

C H A P T E R

16

TEMPLATES AND GENERIC PROGRAMMING

CONTENTS

---

Section 16.1 Defining a Template . . . . .

Section 16.2 Template Argument Deduction . . . . .

Section 16.3 Overloading and Templates . . . . .

Section 16.4 Variadic Templates . . . . .

Section 16.5 Template Specializations . . . . .

Chapter Summary . . . . .

Defined Terms . . . . .

652

678

694

699

706

713

713

Both object-oriented programming (OOP) and generic programming deal with types that are not known at the time the program is written. The distinction between the two is that OOP deals with types that are not known until run time, whereas in generic programming the types become known during compilation.

The containers, iterators, and algorithms described in Part II are all examples of generic programming. When we write a generic program, we write the code in a way that is independent of any particular type. When we use a generic program, we supply the type(s) or value(s) on which that instance of the program will operate.

For example, the library provides a single, generic definition of each container, such as `vector`. We can use that generic definition to define many different types of vectors, each of which differs from the others as to the type of elements the `vector` contains.

Templates are the foundation of generic programming. We can use and have used templates without understanding how they are defined. In this chapter we'll see how to define our own templates.

*Templates* are the foundation for generic programming in C++. A template is a blueprint or formula for creating classes or functions. When we use a generic type, such as `vector`, or a generic function, such as `find`, we supply the information needed to transform that blueprint into a specific class or function. That transformation happens during compilation. In Chapter 3 and Part II we learned how to use templates. In this chapter we'll learn how to define them.

## 16.1 Defining a Template

Imagine that we want to write a function to compare two values and indicate whether the first is less than, equal to, or greater than the second. In practice, we'd want to define several such functions, each of which will compare values of a given type. Our first attempt might be to define several overloaded functions:

```
// returns 0 if the values are equal, -1 if v1 is smaller, 1 if v2 is smaller
int compare(const string &v1, const string &v2)
{
    if (v1 < v2) return -1;
    if (v2 < v1) return 1;
    return 0;
}

int compare(const double &v1, const double &v2)
{
    if (v1 < v2) return -1;
    if (v2 < v1) return 1;
    return 0;
}
```

These functions are nearly identical: The only difference between them is the type of their parameters. The function body is the same in each function.

Having to repeat the body of the function for each type that we compare is tedious and error-prone. More importantly, we need to know when we write the program all the types that we might ever want to compare. This strategy cannot work if we want to be able to use the function on types that our users might supply.



### 16.1.1 Function Templates

Rather than defining a new function for each type, we can define a **function template**. A function template is a formula from which we can generate type-specific versions of that function. The template version of `compare` looks like

```
template <typename T>
int compare(const T &v1, const T &v2)
{
    if (v1 < v2) return -1;
    if (v2 < v1) return 1;
    return 0;
}
```

A template definition starts with the keyword `template` followed by a **template parameter list**, which is a comma-separated list of one or more **template parameters** bracketed by the less-than (<) and greater-than (>) tokens.



In a template definition, the template parameter list cannot be empty.

The template parameter list acts much like a function parameter list. A function parameter list defines local variable(s) of a specified type but does not say how to initialize them. At run time, arguments are supplied that initialize the parameters.

Analogously, template parameters represent types or values used in the definition of a class or function. When we use a template, we specify—either implicitly or explicitly—**template argument(s)** to bind to the template parameter(s).

Our `compare` function declares one type parameter named `T`. Inside `compare`, we use the name `T` to refer to a type. Which *actual type* `T` represents is determined at compile time based on how `compare` is used.

## Instantiating a Function Template

When we call a function template, the compiler (ordinarily) uses the arguments of the call to deduce the template argument(s) for us. That is, when we call `compare`, the compiler uses the type of the arguments to determine what type to bind to the template parameter `T`. For example, in this call

```
cout << compare(1, 0) << endl;           // T is int
```

the arguments have type `int`. The compiler will deduce `int` as the template argument and will bind that argument to the template parameter `T`.

The compiler uses the deduced template parameter(s) to **instantiate** a specific version of the function for us. When the compiler instantiates a template, it creates a new “instance” of the template using the actual template argument(s) in place of the corresponding template parameter(s). For example, given the calls

```
// instantiates int compare(const int&, const int&)
cout << compare(1, 0) << endl;           // T is int

// instantiates int compare(const vector<int>&, const vector<int>&)
vector<int> vec1{1, 2, 3}, vec2{4, 5, 6};
cout << compare(vec1, vec2) << endl;    // T is vector<int>
```

the compiler will instantiate two different versions of `compare`. For the first call, the compiler will write and compile a version of `compare` with `T` replaced by `int`:

```
int compare(const int &v1, const int &v2)
{
    if (v1 < v2) return -1;
    if (v2 < v1) return 1;
    return 0;
}
```

For the second call, it will generate a version of `compare` with `T` replaced by `vector<int>`. These compiler-generated functions are generally referred to as an **instantiation** of the template.

## Template Type Parameters

Our `compare` function has one template **type parameter**. In general, we can use a type parameter as a type specifier in the same way that we use a built-in or class type specifier. In particular, a type parameter can be used to name the return type or a function parameter type, and for variable declarations or casts inside the function body:

```
// ok: same type used for the return type and parameter
template <typename T> T foo(T* p)
{
    T tmp = *p; // tmp will have the type to which p points
    // ...
    return tmp;
}
```

Each type parameter must be preceded by the keyword `class` or `typename`:

```
// error: must precede U with either typename or class
template <typename T, U> T calc(const T&, const U&);
```

These keywords have the same meaning and can be used interchangeably inside a template parameter list. A template parameter list can use both keywords:

```
// ok: no distinction between typename and class in a template parameter list
template <typename T, class U> calc (const T&, const U&);
```

It may seem more intuitive to use the keyword `typename` rather than `class` to designate a template type parameter. After all, we can use built-in (nonclass) types as a template type argument. Moreover, `typename` more clearly indicates that the name that follows is a type name. However, `typename` was added to C++ after templates were already in widespread use; some programmers continue to use `class` exclusively.

## Nontype Template Parameters

In addition to defining type parameters, we can define templates that take **nontype parameters**. A nontype parameter represents a value rather than a type. Nontype parameters are specified by using a specific type name instead of the `class` or `typename` keyword.

When the template is instantiated, nontype parameters are replaced with a value supplied by the user or deduced by the compiler. These values must be constant expressions (§ 2.4.4, p. 65), which allows the compiler to instantiate the templates during compile time.

As an example, we can write a version of `compare` that will handle string literals. Such literals are arrays of `const char`. Because we cannot copy an array, we'll define our parameters as references to an array (§ 6.2.4, p. 217). Because we'd like to be able to compare literals of different lengths, we'll give our template two nontype parameters. The first template parameter will represent the size of the first array, and the second parameter will represent the size of the second array:

```
template<unsigned N, unsigned M>
int compare(const char (&p1)[N], const char (&p2)[M])
{
    return strcmp(p1, p2);
}
```

When we call this version of `compare`:

```
compare("hi", "mom")
```

the compiler will use the size of the literals to instantiate a version of the template with the sizes substituted for `N` and `M`. Remembering that the compiler inserts a null terminator at the end of a string literal (§ 2.1.3, p. 39), the compiler will instantiate

```
int compare(const char (&p1)[3], const char (&p2)[4])
```

A nontype parameter may be an integral type, or a pointer or (lvalue) reference to an object or to a function type. An argument bound to a nontype integral parameter must be a constant expression. Arguments bound to a pointer or reference nontype parameter must have static lifetime (Chapter 12, p. 450). We may not use an ordinary (nonstatic) local object or a dynamic object as a template argument for reference or pointer nontype template parameters. A pointer parameter can also be instantiated by `nullptr` or a zero-valued constant expression.

A template nontype parameter is a constant value inside the template definition. A nontype parameter can be used when constant expressions are required, for example, to specify the size of an array.



Template arguments used for nontype template parameters must be constant expressions.

## inline and constexpr Function Templates

A function template can be declared `inline` or `constexpr` in the same ways as nontemplate functions. The `inline` or `constexpr` specifier follows the template parameter list and precedes the return type:

```
// ok: inline specifier follows the template parameter list
template <typename T> inline T min(const T&, const T&);

// error: incorrect placement of the inline specifier
inline template <typename T> T min(const T&, const T&);
```

## Writing Type-Independent Code



Simple though it is, our initial `compare` function illustrates two important principles for writing generic code:

- The function parameters in the template are references to `const`.
- The tests in the body use only `<` comparisons.

By making the function parameters references to `const`, we ensure that our function can be used on types that cannot be copied. Most types—including the built-in types and, except for `unique_ptr` and the IO types, all the library types we’ve used—do allow copying. However, there can be class types that do not allow copying. By making our parameters references to `const`, we ensure that such types can be used with our `compare` function. Moreover, if `compare` is called with large objects, then this design will also make the function run faster.

You might think it would be more natural for the comparisons to be done using both the `<` and `>` operators:

```
// expected comparison
if (v1 < v2) return -1;
if (v1 > v2) return 1;
return 0;
```

However, by writing the code using only the `<` operator, we reduce the requirements on types that can be used with our `compare` function. Those types must support `<`, but they need not also support `>`.

In fact, if we were truly concerned about type independence and portability, we probably should have defined our function using the `less` (§ 14.8.2, p. 575):

```
// version of compare that will be correct even if used on pointers; see § 14.8.2 (p. 575)
template <typename T> int compare(const T &v1, const T &v2)
{
    if (less<T>() (v1, v2)) return -1;
    if (less<T>() (v2, v1)) return 1;
    return 0;
}
```

The problem with our original version is that if a user calls it with two pointers and those pointers do not point to the same array, then our code is undefined.



Template programs should try to minimize the number of requirements placed on the argument types.



## Template Compilation

When the compiler sees the definition of a template, it does not generate code. It generates code only when we instantiate a specific instance of the template. The fact that code is generated only when we use a template (and not when we define it) affects how we organize our source code and when errors are detected.

Ordinarily, when we call a function, the compiler needs to see only a declaration for the function. Similarly, when we use objects of class type, the class definition must be available, but the definitions of the member functions need not be present. As a result, we put class definitions and function declarations in header files and definitions of ordinary and class-member functions in source files.

Templates are different: To generate an instantiation, the compiler needs to have the code that defines a function template or class template member function. As a result, unlike nontemplate code, headers for templates typically include definitions as well as declarations



Definitions of function templates and member functions of class templates are ordinarily put into header files.

### KEY CONCEPT: TEMPLATES AND HEADERS

Templates contain two kinds of names:

- Those that do not depend on a template parameter
- Those that do depend on a template parameter

It is up to the provider of a template to ensure that all names that do not depend on a template parameter are visible when the template is used. Moreover, the template provider must ensure that the definition of the template, including the definitions of the members of a class template, are visible when the template is instantiated.

It is up to *users* of a template to ensure that declarations for all functions, types, and operators associated with the types used to instantiate the template are visible.

Both of these requirements are easily satisfied by well-structured programs that make appropriate use of headers. Authors of templates should provide a header that contains the template definition along with declarations for all the names used in the class template or in the definitions of its members. Users of the template must include the header for the template and for any types used to instantiate that template.

## Compilation Errors Are Mostly Reported during Instantiation

The fact that code is not generated until a template is instantiated affects when we learn about compilation errors in the code inside the template. In general, there are three stages during which the compiler might flag an error.

The first stage is when we compile the template itself. The compiler generally can't find many errors at this stage. The compiler can detect syntax errors—such as forgetting a semicolon or misspelling a variable name—but not much else.

The second error-detection time is when the compiler sees a use of the template. At this stage, there is still not much the compiler can check. For a call to a function template, the compiler typically will check that the number of the arguments is appropriate. It can also detect whether two arguments that are supposed to have the same type do so. For a class template, the compiler can check that the right number of template arguments are provided but not much more.

The third time when errors are detected is during instantiation. It is only then that type-related errors can be found. Depending on how the compiler manages instantiation, these errors may be reported at link time.

When we write a template, the code may not be overtly type specific, but template code usually makes some assumptions about the types that will be used. For example, the code inside our original `compare` function:

```
if (v1 < v2) return -1; // requires < on objects of type T
if (v2 < v1) return 1;  // requires < on objects of type T
return 0;              // returns int; not dependent on T
```

assumes that the argument type has a `<` operator. When the compiler processes the body of this template, it cannot verify whether the conditions in the `if` statements are legal. If the arguments passed to `compare` have a `<` operation, then the code is fine, but not otherwise. For example,

```
Sales_data data1, data2;
cout << compare(data1, data2) << endl; // error: no < on Sales_data
```

This call instantiates a version of `compare` with `T` replaced by `Sales_data`. The `if` conditions attempt to use `<` on `Sales_data` objects, but there is no such operator. This instantiation generates a version of the function that will not compile. However, errors such as this one cannot be detected until the compiler instantiates the definition of `compare` on type `Sales_data`.



**WARNING**

It is up to the caller to guarantee that the arguments passed to the template support any operations that template uses, and that those operations behave correctly in the context in which the template uses them.

## EXERCISES SECTION 16.1.1

**Exercise 16.1:** Define instantiation.

**Exercise 16.2:** Write and test your own versions of the `compare` functions.

**Exercise 16.3:** Call your `compare` function on two `Sales_data` objects to see how your compiler handles errors during instantiation.

**Exercise 16.4:** Write a template that acts like the library `find` algorithm. The function will need two template type parameters, one to represent the function's iterator parameters and the other for the type of the value. Use your function to find a given value in a `vector<int>` and in a `list<string>`.

**Exercise 16.5:** Write a template version of the `print` function from § 6.2.4 (p. 217) that takes a reference to an array and can handle arrays of any size and any element type.

**Exercise 16.6:** How do you think the library `begin` and `end` functions that take an array argument work? Define your own versions of these functions.

**Exercise 16.7:** Write a `constexpr` template that returns the size of a given array.

**Exercise 16.8:** In the “Key Concept” box on page 108, we noted that as a matter of habit C++ programmers prefer using `!=` to using `<`. Explain the rationale for this habit.



## 16.1.2 Class Templates

A **class template** is a blueprint for generating classes. Class templates differ from function templates in that the compiler cannot deduce the template parameter type(s) for a class template. Instead, as we've seen many times, to use a class template we must supply additional information inside angle brackets following



the template's name (§ 3.3, p. 97). That extra information is the list of template arguments to use in place of the template parameters.

## Defining a Class Template

As an example, we'll implement a template version of `StrBlob` (§ 12.1.1, p. 456). We'll name our template `Blob` to indicate that it is no longer specific to strings. Like `StrBlob`, our template will provide shared (and checked) access to the elements it holds. Unlike that class, our template can be used on elements of pretty much any type. As with the library containers, our users will have to specify the element type when they use a `Blob`.

Like function templates, class templates begin with the keyword `template` followed by a template parameter list. In the definition of the class template (and its members), we use the template parameters as stand-ins for types or values that will be supplied when the template is used:

```
template <typename T> class Blob {
public:
    typedef T value_type;
    typedef typename std::vector<T>::size_type size_type;
    // constructors
    Blob();
    Blob(std::initializer_list<T> il);
    // number of elements in the Blob
    size_type size() const { return data->size(); }
    bool empty() const { return data->empty(); }
    // add and remove elements
    void push_back(const T &t) { data->push_back(t); }
    // move version; see § 13.6.3 (p. 548)
    void push_back(T &&t) { data->push_back(std::move(t)); }
    void pop_back();
    // element access
    T& back();
    T& operator[] (size_type i); // defined in § 14.5 (p. 566)
private:
    std::shared_ptr<std::vector<T>> data;
    // throws msg if data[i] isn't valid
    void check(size_type i, const std::string &msg) const;
};
```

Our `Blob` template has one template type parameter, named `T`. We use the type parameter anywhere we refer to the element type that the `Blob` holds. For example, we define the return type of the operations that provide access to the elements in the `Blob` as `T&`. When a user instantiates a `Blob`, these uses of `T` will be replaced by the specified template argument type.

With the exception of the template parameter list, and the use of `T` instead of `string`, this class is the same as the version we defined in § 12.1.1 (p. 456) and updated in § 12.1.6 (p. 475) and in Chapters 13 and 14.

## Instantiating a Class Template

As we've seen many times, when we use a class template, we must supply extra information. We can now see that that extra information is a list of **explicit template arguments** that are bound to the template's parameters. The compiler uses these template arguments to instantiate a specific class from the template.

For example, to define a type from our `Blob` template, we must provide the element type:

```
Blob<int> ia; // empty Blob<int>
Blob<int> ia2 = {0,1,2,3,4}; // Blob<int> with five elements
```

Both `ia` and `ia2` use the same type-specific version of `Blob` (i.e., `Blob<int>`). From these definitions, the compiler will instantiate a class that is equivalent to

```
template <> class Blob<int> {
    typedef typename std::vector<int>::size_type size_type;
    Blob();
    Blob(std::initializer_list<int> il);
    // ...
    int& operator[] (size_type i);
private:
    std::shared_ptr<std::vector<int>> data;
    void check(size_type i, const std::string &msg) const;
};
```

When the compiler instantiates a class from our `Blob` template, it rewrites the `Blob` template, replacing each instance of the template parameter `T` by the given template argument, which in this case is `int`.

The compiler generates a different class for each element type we specify:

```
// these definitions instantiate two distinct Blob types
Blob<string> names; // Blob that holds strings
Blob<double> prices; // different element type
```

These definitions would trigger instantiations of two distinct classes: The definition of `names` creates a `Blob` class in which each occurrence of `T` is replaced by `string`. The definition of `prices` generates a `Blob` with `T` replaced by `double`.



Each instantiation of a class template constitutes an independent class. The type `Blob<string>` has no relationship to, or any special access to, the members of any other `Blob` type.



## References to a Template Type in the Scope of the Template

In order to read template class code, it can be helpful to remember that the name of a class template is not the name of a type (§ 3.3, p. 97). A class template is used to instantiate a type, and an instantiated type always includes template argument(s).

What can be confusing is that code in a class template generally doesn't use the name of an actual type (or value) as a template argument. Instead, we often use the template's own parameter(s) as the template argument(s). For example, our

data member uses two templates, `vector` and `shared_ptr`. Whenever we use a template, we must supply template arguments. In this case, the template argument we supply is the same type that is used to instantiate the `Blob`. Therefore, the definition of data

```
std::shared_ptr<std::vector<T>> data;
```

uses `Blob`'s type parameter to say that data is the instantiation of `shared_ptr` that points to the instantiation of `vector` that holds objects of type `T`. When we instantiate a particular kind of `Blob`, such as `Blob<string>`, then data will be

```
shared_ptr<vector<string>>
```

If we instantiate `Blob<int>`, then data will be `shared_ptr<vector<int>>`, and so on.

## Member Functions of Class Templates

As with any class, we can define the member functions of a class template either inside or outside of the class body. As with any other class, members defined inside the class body are implicitly inline.

A class template member function is itself an ordinary function. However, each instantiation of the class template has its own version of each member. As a result, a member function of a class template has the same template parameters as the class itself. Therefore, a member function defined outside the class template body starts with the keyword `template` followed by the class' template parameter list.

As usual, when we define a member outside its class, we must say to which class the member belongs. Also as usual, the name of a class generated from a template includes its template arguments. When we define a member, the template argument(s) are the same as the template parameter(s). That is, for a given member function of `StrBlob` that was defined as

```
ret-type StrBlob::member-name (parm-list)
```

the corresponding `Blob` member will look like

```
template <typename T>
ret-type Blob<T>::member-name (parm-list)
```

## The check and Element Access Members

We'll start by defining the `check` member, which verifies a given index:

```
template <typename T>
void Blob<T>::check(size_type i, const std::string &msg) const
{
    if (i >= data->size())
        throw std::out_of_range(msg);
}
```

Aside from the differences in the class name and the use of the template parameter list, this function is identical to the original `StrBlob` member.

The subscript operator and back function use the template parameter to specify the return type but are otherwise unchanged:

```

template <typename T>
T& Blob<T>::back()
{
    check(0, "back on empty Blob");
    return data->back();
}
template <typename T>
T& Blob<T>::operator[] (size_type i)
{
    // if i is too big, check will throw, preventing access to a nonexistent element
    check(i, "subscript out of range");
    return (*data)[i];
}

```

In our original `StrBlob` class these operators returned `string&`. The template versions will return a reference to whatever type is used to instantiate `Blob`.

The `pop_back` function is nearly identical to our original `StrBlob` member:

```

template <typename T> void Blob<T>::pop_back()
{
    check(0, "pop_back on empty Blob");
    data->pop_back();
}

```

The subscript operator and back members are overloaded on `const`. We leave the definition of these members, and of the front members, as an exercise.

## Blob Constructors

As with any other member defined outside a class template, a constructor starts by declaring the template parameters for the class template of which it is a member:

```

template <typename T>
Blob<T>::Blob(): data(std::make_shared<std::vector<T>>()) { }

```

Here we are defining the member named `Blob` in the scope of `Blob<T>`. Like our `StrBlob` default constructor (§ 12.1.1, p. 456), this constructor allocates an empty vector and stores the pointer to that vector in `data`. As we've seen, we use the class' own type parameter as the template argument of the vector we allocate.

Similarly, the constructor that takes an `initializer_list` uses its type parameter `T` as the element type for its `initializer_list` parameter:

```

template <typename T>
Blob<T>::Blob(std::initializer_list<T> il):
    data(std::make_shared<std::vector<T>>(il)) { }

```

Like the default constructor, this constructor allocates a new vector. In this case, we initialize that vector from the parameter, `il`.

To use this constructor, we must pass an `initializer_list` in which the elements are compatible with the element type of the `Blob`:

```

Blob<string> articles = {"a", "an", "the"};

```

The parameter in this constructor has type `initializer_list<string>`. Each string literal in the list is implicitly converted to `string`.

## Instantiation of Class-Template Member Functions

By default, a member function of a class template is instantiated *only* if the program uses that member function. For example, this code

```
// instantiates Blob<int> and the initializer_list<int> constructor
Blob<int> squares = {0,1,2,3,4,5,6,7,8,9};
// instantiates Blob<int>::size() const
for (size_t i = 0; i != squares.size(); ++i)
    squares[i] = i*i; // instantiates Blob<int>::operator[] (size_t)
```

instantiates the `Blob<int>` class and three of its member functions: `operator[]`, `size`, and the `initializer_list<int>` constructor.

If a member function isn't used, it is not instantiated. The fact that members are instantiated only if we use them lets us instantiate a class with a type that may not meet the requirements for some of the template's operations (§ 9.2, p. 329).



By default, a member of an instantiated class template is instantiated only if the member is used.

## Simplifying Use of a Template Class Name inside Class Code

There is one exception to the rule that we must supply template arguments when we use a class template type. Inside the scope of the class template itself, we may use the name of the template without arguments:

```
// BlobPtr throws an exception on attempts to access a nonexistent element
template <typename T> class BlobPtr
public:
    BlobPtr(): curr(0) { }
    BlobPtr(Blob<T> &a, size_t sz = 0):
        wptr(a.data), curr(sz) { }
    T& operator*() const
    { auto p = check(curr, "dereference past end");
      return (*p)[curr]; // (*p) is the vector to which this object points
    }
    // increment and decrement
    BlobPtr& operator++(); // prefix operators
    BlobPtr& operator--();
private:
    // check returns a shared_ptr to the vector if the check succeeds
    std::shared_ptr<std::vector<T>>
        check(std::size_t, const std::string&) const;
    // store a weak_ptr, which means the underlying vector might be destroyed
    std::weak_ptr<std::vector<T>> wptr;
    std::size_t curr; // current position within the array
};
```

Careful readers will have noted that the prefix increment and decrement members of `BlobPtr` return `BlobPtr&`, not `BlobPtr<T>&`. When we are inside the scope of a class template, the compiler treats references to the template itself as if we had

supplied template arguments matching the template's own parameters. That is, it is as if we had written:

```
BlobPtr<T>& operator++();
BlobPtr<T>& operator--();
```

## Using a Class Template Name outside the Class Template Body

When we define members outside the body of a class template, we must remember that we are not in the scope of the class until the class name is seen (§ 7.4, p. 282):

```
// postfix: increment/decrement the object but return the unchanged value
template <typename T>
BlobPtr<T> BlobPtr<T>::operator++(int)
{
    // no check needed here; the call to prefix increment will do the check
    BlobPtr ret = *this;    // save the current value
    ++*this;               // advance one element; prefix ++ checks the increment
    return ret;            // return the saved state
}
```

Because the return type appears outside the scope of the class, we must specify that the return type returns a `BlobPtr` instantiated with the same type as the class. Inside the function body, we are in the scope of the class so do not need to repeat the template argument when we define `ret`. When we do not supply template arguments, the compiler assumes that we are using the same type as the member's instantiation. Hence, the definition of `ret` is as if we had written:

```
BlobPtr<T> ret = *this;
```



Inside the scope of a class template, we may refer to the template without specifying template argument(s).

## Class Templates and Friends

When a class contains a friend declaration (§ 7.2.1, p. 269), the class and the friend can independently be templates or not. A class template that has a nontemplate friend grants that friend access to all the instantiations of the template. When the friend is itself a template, the class granting friendship controls whether friendship includes all instantiations of the template or only specific instantiation(s).

## One-to-One Friendship

The most common form of friendship from a class template to another template (class or function) establishes friendship between corresponding instantiations of the class and its friend. For example, our `Blob` class should declare the `BlobPtr` class and a template version of the `Blob` equality operator (originally defined for `StrBlob` in the exercises in § 14.3.1 (p. 562)) as friends.

In order to refer to a specific instantiation of a template (class or function) we must first declare the template itself. A template declaration includes the template's template parameter list:

```
// forward declarations needed for friend declarations in Blob
template <typename> class BlobPtr;
template <typename> class Blob; // needed for parameters in operator==
template <typename T>
    bool operator==(const Blob<T>&, const Blob<T>&);
template <typename T> class Blob {
    // each instantiation of Blob grants access to the version of
    // BlobPtr and the equality operator instantiated with the same type
    friend class BlobPtr<T>;
    friend bool operator==(T>
        (const Blob<T>&, const Blob<T>&);
    // other members as in § 12.1.1 (p. 456)
};
```

We start by declaring that `Blob`, `BlobPtr`, and `operator==` are templates. These declarations are needed for the parameter declaration in the `operator==` function and the friend declarations in `Blob`.

The friend declarations use `Blob`'s template parameter as their own template argument. Thus, the friendship is restricted to those instantiations of `BlobPtr` and the equality operator that are instantiated with the same type:

```
Blob<char> ca; // BlobPtr<char> and operator==<char> are friends
Blob<int> ia; // BlobPtr<int> and operator==<int> are friends
```

The members of `BlobPtr<char>` may access the nonpublic parts of `ca` (or any other `Blob<char>` object), but `ca` has no special access to `ia` (or any other `Blob<int>`) or to any other instantiation of `Blob`.

## General and Specific Template Friendship

A class can also make every instantiation of another template its friend, or it may limit friendship to a specific instantiation:

```
// forward declaration necessary to befriend a specific instantiation of a template
template <typename T> class Pal;
class C { // C is an ordinary, nontemplate class
    friend class Pal<C>; // Pal instantiated with class C is a friend to C
    // all instances of Pal2 are friends to C;
    // no forward declaration required when we befriend all instantiations
    template <typename T> friend class Pal2;
};

template <typename T> class C2 { // C2 is itself a class template
    // each instantiation of C2 has the same instance of Pal as a friend
    friend class Pal<T>; // a template declaration for Pal must be in scope
    // all instances of Pal2 are friends of each instance of C2, prior declaration needed
    template <typename X> friend class Pal2;
    // Pal3 is a nontemplate class that is a friend of every instance of C2
    friend class Pal3; // prior declaration for Pal3 not needed
};
```

To allow all instantiations as friends, the friend declaration must use template parameter(s) that differ from those used by the class itself.

## Befriending the Template's Own Type Parameter

**C++  
11**

Under the new standard, we can make a template type parameter a friend:

```
template <typename Type> class Bar {
    friend Type; // grants access to the type used to instantiate Bar
    // ...
};
```

Here we say that whatever type is used to instantiate `Bar` is a friend. Thus, for some type named `Foo`, `Foo` would be a friend of `Bar<Foo>`, `Sales_data` a friend of `Bar<Sales_data>`, and so on.

It is worth noting that even though a friend ordinarily must be a class or a function, it is okay for `Bar` to be instantiated with a built-in type. Such friendship is allowed so that we can instantiate classes such as `Bar` with built-in types.

## Template Type Aliases

An instantiation of a class template defines a class type, and as with any other class type, we can define a typedef (§ 2.5.1, p. 67) that refers to that instantiated class:

```
typedef Blob<string> StrBlob;
```

This typedef will let us run the code we wrote in § 12.1.1 (p. 456) using our template version of `Blob` instantiated with `string`. Because a template is not a type, we cannot define a typedef that refers to a template. That is, there is no way to define a typedef that refers to `Blob<T>`.

**C++  
11**

However, the new standard lets us define a type alias for a class template:

```
template<typename T> using twin = pair<T, T>;
twin<string> authors; // authors is a pair<string, string>
```

Here we've defined `twin` as a synonym for pairs in which the members have the same type. Users of `twin` need to specify that type only once.

A template type alias is a synonym for a family of classes:

```
twin<int> win_loss; // win_loss is a pair<int, int>
twin<double> area; // area is a pair<double, double>
```

Just as we do when we use a class template, when we use `twin`, we specify which particular kind of `twin` we want.

When we define a template type alias, we can fix one or more of the template parameters:

```
template <typename T> using partNo = pair<T, unsigned>;
partNo<string> books; // books is a pair<string, unsigned>
partNo<Vehicle> cars; // cars is a pair<Vehicle, unsigned>
partNo<Student> kids; // kids is a pair<Student, unsigned>
```

Here we've defined `partNo` as a synonym for the family of types that are pairs in which the second member is an unsigned. Users of `partNo` specify a type for the first member of the pair but have no choice about second.



## static Members of Class Templates

Like any other class, a class template can declare static members (§ 7.6, p. 300):

```
template <typename T> class Foo {
public:
    static std::size_t count() { return ctr; }
    // other interface members
private:
    static std::size_t ctr;
    // other implementation members
};
```

Here `Foo` is a class template that has a public static member function named `count` and a private static data member named `ctr`. Each instantiation of `Foo` has its own instance of the static members. That is, for any given type `X`, there is one `Foo<X>::ctr` and one `Foo<X>::count` member. All objects of type `Foo<X>` share the same `ctr` object and `count` function. For example,

```
// instantiates static members Foo<string>::ctr and Foo<string>::count
Foo<string> fs;
// all three objects share the same Foo<int>::ctr and Foo<int>::count members
Foo<int> fi, fi2, fi3;
```

As with any other static data member, there must be exactly one definition of each static data member of a template class. However, there is a distinct object for each instantiation of a class template. As a result, we define a static data member as a template similarly to how we define the member functions of that template:

```
template <typename T>
size_t Foo<T>::ctr = 0; // define and initialize ctr
```

As with any other member of a class template, we start by defining the template parameter list, followed by the type of the member we are defining and the member's name. As usual, a member's name includes the member's class name, which for a class generated from a template includes its template arguments. Thus, when `Foo` is instantiated for a particular template argument type, a separate `ctr` will be instantiated for that class type and initialized to 0.

As with static members of nontemplate classes, we can access a static member of a class template through an object of the class type or by using the scope operator to access the member directly. Of course, to use a static member through the class, we must refer to a specific instantiation:

```
Foo<int> fi; // instantiates Foo<int> class
           // and the static data member ctr

auto ct = Foo<int>::count(); // instantiates Foo<int>::count
ct = fi.count();           // uses Foo<int>::count
ct = Foo::count();         // error: which template instantiation?
```

Like any other member function, a static member function is instantiated only if it is used in a program.

## EXERCISES SECTION 16.1.2

**Exercise 16.9:** What is a function template? What is a class template?

**Exercise 16.10:** What happens when a class template is instantiated?

**Exercise 16.11:** The following definition of `List` is incorrect. How would you fix it?

```
template <typename elemType> class ListItem;
template <typename elemType> class List {
public:
    List<elemType>();
    List<elemType>(const List<elemType> &);
    List<elemType>& operator=(const List<elemType> &);
    ~List();
    void insert(ListItem *ptr, elemType value);
private:
    ListItem *front, *end;
};
```

**Exercise 16.12:** Write your own version of the `Blob` and `BlobPtr` templates, including the various `const` members that were not shown in the text.

**Exercise 16.13:** Explain which kind of friendship you chose for the equality and relational operators for `BlobPtr`.

**Exercise 16.14:** Write a `Screen` class template that uses nontype parameters to define the height and width of the `Screen`.

**Exercise 16.15:** Implement input and output operators for your `Screen` template. Which, if any, friends are necessary in class `Screen` to make the input and output operators work? Explain why each friend declaration, if any, was needed.

**Exercise 16.16:** Rewrite the `StrVec` class (§ 13.5, p. 526) as a template named `Vec`.



### 16.1.3 Template Parameters

Like the names of function parameters, a template parameter name has no intrinsic meaning. We ordinarily name type parameters `T`, but we can use any name:

```
template <typename Foo> Foo calc(const Foo& a, const Foo& b)
{
    Foo tmp = a; // tmp has the same type as the parameters and return type
    // ...
    return tmp; // return type and parameters have the same type
}
```

### Template Parameters and Scope

Template parameters follow normal scoping rules. The name of a template parameter can be used after it has been declared and until the end of the template declaration or definition. As with any other name, a template parameter hides any

declaration of that name in an outer scope. Unlike most other contexts, however, a name used as a template parameter may not be reused within the template:

```
typedef double A;
template <typename A, typename B> void f(A a, B b)
{
    A tmp = a; // tmp has same type as the template parameter A, not double
    double B; // error: redeclares template parameter B
}
```

Normal name hiding says that the typedef of A is hidden by the type parameter named A. Thus, tmp is not a double; it has whatever type gets bound to the template parameter A when calc is used. Because we cannot reuse names of template parameters, the declaration of the variable named B is an error.

Because a parameter name cannot be reused, the name of a template parameter can appear only once within a given template parameter list:

```
// error: illegal reuse of template parameter name V
template <typename V, typename V> // ...
```

## Template Declarations

A template declaration must include the template parameters:

```
// declares but does not define compare and Blob
template <typename T> int compare(const T&, const T&);
template <typename T> class Blob;
```

As with function parameters, the names of a template parameter need not be the same across the declaration(s) and the definition of the same template:

```
// all three uses of calc refer to the same function template
template <typename T> T calc(const T&, const T&); // declaration
template <typename U> U calc(const U&, const U&); // declaration
// definition of the template
template <typename Type>
Type calc(const Type& a, const Type& b) { /* ... */ }
```

Of course, every declaration and the definition of a given template must have the same number and kind (i.e., type or nontype) of parameters.



For reasons we'll explain in § 16.3 (p. 698), declarations for all the templates needed by a given file usually should appear together at the beginning of a file before any code that uses those names.

## Using Class Members That Are Types

Recall that we use the scope operator (::) to access both static members and type members (§ 7.4, p. 282, and § 7.6, p. 301). In ordinary (nontemplate) code, the compiler has access to the class definition. As a result, it knows whether a name accessed through the scope operator is a type or a static member. For example,

when we write `string::size_type`, the compiler has the definition of `string` and can see that `size_type` is a type.

Assuming `T` is a template type parameter, When the compiler sees code such as `T::mem` it won't know until instantiation time whether `mem` is a type or a static data member. However, in order to process the template, the compiler must know whether a name represents a type. For example, assuming `T` is the name of a type parameter, when the compiler sees a statement of the following form:

```
T::size_type * p;
```

it needs to know whether we're defining a variable named `p` or are multiplying a static data member named `size_type` by a variable named `p`.

By default, the language assumes that a name accessed through the scope operator is not a type. As a result, if we want to use a type member of a template type parameter, we must explicitly tell the compiler that the name is a type. We do so by using the keyword `typename`:

```
template <typename T>
typename T::value_type top(const T& c)
{
    if (!c.empty())
        return c.back();
    else
        return typename T::value_type();
}
```

Our `top` function expects a container as its argument and uses `typename` to specify its return type and to generate a value initialized element (§ 7.5.3, p. 293) to return if `c` has no elements.



When we want to inform the compiler that a name represents a type, we must use the keyword `typename`, not `class`.

## Default Template Arguments

C++  
11

Just as we can supply default arguments to function parameters (§ 6.5.1, p. 236), we can also supply **default template arguments**. Under the new standard, we can supply default arguments for both function and class templates. Earlier versions of the language, allowed default arguments only with class templates.

As an example, we'll rewrite `compare` to use the library `less` function-object template (§ 14.8.2, p. 574) by default:

```
// compare has a default template argument, less<T>
// and a default function argument, F()
template <typename T, typename F = less<T>>
int compare(const T &v1, const T &v2, F f = F())
{
    if (f(v1, v2)) return -1;
    if (f(v2, v1)) return 1;
    return 0;
}
```

Here we've given our template a second type parameter, named `F`, that represents the type of a callable object (§ 10.3.2, p. 388) and defined a new function parameter, `f`, that will be bound to a callable object.

We've also provided defaults for this template parameter and its corresponding function parameter. The default template argument specifies that `compare` will use the library `less` function-object class, instantiated with the same type parameter as `compare`. The default function argument says that `f` will be a default-initialized object of type `F`.

When users call this version of `compare`, they may supply their own comparison operation but are not required to do so:

```
bool i = compare(0, 42); // uses less; i is -1
// result depends on the isbns in item1 and item2
Sales_data item1(cin), item2(cin);
bool j = compare(item1, item2, compareIsbn);
```

The first call uses the default function argument, which is a default-initialized object of type `less<T>`. In this call, `T` is `int` so that object has type `less<int>`. This instantiation of `compare` will use `less<int>` to do its comparisons.

In the second call, we pass `compareIsbn` (§ 11.2.2, p. 425) and two objects of type `Sales_data`. When `compare` is called with three arguments, the type of the third argument must be a callable object that returns a type that is convertible to `bool` and takes arguments of a type compatible with the types of the first two arguments. As usual, the types of the template parameters are deduced from their corresponding function arguments. In this call, the type of `T` is deduced as `Sales_data` and `F` is deduced as the type of `compareIsbn`.

As with function default arguments, a template parameter may have a default argument only if all of the parameters to its right also have default arguments.

## Template Default Arguments and Class Templates

Whenever we use a class template, we must always follow the template's name with brackets. The brackets indicate that a class must be instantiated from a template. In particular, if a class template provides default arguments for all of its template parameters, and we want to use those defaults, we must put an empty bracket pair following the template's name:

```
template <class T = int> class Numbers { // by default T is int
public:
    Numbers(T v = 0): val(v) { }
    // various operations on numbers
private:
    T val;
};

Numbers<long double> lots_of_precision;
Numbers<> average_precision; // empty <> says we want the default type
```

Here we instantiate two versions of `Numbers`: `average_precision` instantiates `Numbers` with `T` replaced by `int`; `lots_of_precision` instantiates `Numbers` with `T` replaced by `long double`.

### EXERCISES SECTION 16.1.3

**Exercise 16.17:** What, if any, are the differences between a type parameter that is declared as a `typename` and one that is declared as a `class`? When must `typename` be used?

**Exercise 16.18:** Explain each of the following function template declarations and identify whether any are illegal. Correct each error that you find.

- (a) `template <typename T, U, typename V> void f1(T, U, V);`
- (b) `template <typename T> T f2(int &T);`
- (c) `inline template <typename T> T foo(T, unsigned int*);`
- (d) `template <typename T> f4(T, T);`
- (e) `typedef char Ctype;`  
`template <typename Ctype> Ctype f5(Ctype a);`

**Exercise 16.19:** Write a function that takes a reference to a container and prints the elements in that container. Use the container's `size_type` and `size` members to control the loop that prints the elements.

**Exercise 16.20:** Rewrite the function from the previous exercise to use iterators returned from `begin` and `end` to control the loop.

## 16.1.4 Member Templates

A class—either an ordinary class or a class template—may have a member function that is itself a template. Such members are referred to as **member templates**. Member templates may not be virtual.

### Member Templates of Ordinary (Nontemplate) Classes

As an example of an ordinary class that has a member template, we'll define a class that is similar to the default deleter type used by `unique_ptr` (§ 12.1.5, p. 471). Like the default deleter, our class will have an overloaded function-call operator (§ 14.8, p. 571) that will take a pointer and execute `delete` on the given pointer. Unlike the default deleter, our class will also print a message whenever the deleter is executed. Because we want to use our deleter with any type, we'll make the call operator a template:

```
// function-object class that calls delete on a given pointer
class DebugDelete {
public:
    DebugDelete(std::ostream &s = std::cerr): os(s) { }
    // as with any function template, the type of T is deduced by the compiler
    template <typename T> void operator()(T *p) const
    { os << "deleting unique_ptr" << std::endl; delete p; }
private:
    std::ostream &os;
};
```

Like any other template, a member template starts with its own template parameter list. Each `DebugDelete` object has an `ostream` member on which to write, and a member function that is itself a template. We can use this class as a replacement for `delete`:

```
double* p = new double;
DebugDelete d;    // an object that can act like a delete expression
d(p); // calls DebugDelete::operator() (double*), which deletes p

int* ip = new int;
// calls operator() (int*) on a temporary DebugDelete object
DebugDelete() (ip);
```

Because calling a `DebugDelete` object deletes its given pointer, we can also use `DebugDelete` as the deleter of a `unique_ptr`. To override the deleter of a `unique_ptr`, we supply the type of the deleter inside brackets and supply an object of the deleter type to the constructor (§ 12.1.5, p. 471):

```
// destroying the the object to which p points
// instantiates DebugDelete::operator()<int>(int *)
unique_ptr<int, DebugDelete> p(new int, DebugDelete());
// destroying the the object to which sp points
// instantiates DebugDelete::operator()<string>(string*)
unique_ptr<string, DebugDelete> sp(new string, DebugDelete());
```

Here, we've said that `p`'s deleter will have type `DebugDelete`, and we have supplied an unnamed object of that type in `p`'s constructor.

The `unique_ptr` destructor calls the `DebugDelete`'s call operator. Thus, whenever `unique_ptr`'s destructor is instantiated, `DebugDelete`'s call operator will also be instantiated: Thus, the definitions above will instantiate:

```
// sample instantiations for member templates of DebugDelete
void DebugDelete::operator()(int *p) const { delete p; }
void DebugDelete::operator()(string *p) const { delete p; }
```

## Member Templates of Class Templates

We can also define a member template of a class template. In this case, both the class and the member have their own, independent, template parameters.

As an example, we'll give our `Blob` class a constructor that will take two iterators denoting a range of elements to copy. Because we'd like to support iterators into varying kinds of sequences, we'll make this constructor a template:

```
template <typename T> class Blob {
    template <typename It> Blob(It b, It e);
    // ...
};
```

This constructor has its own template type parameter, `It`, which it uses for the type of its two function parameters.

Unlike ordinary function members of class templates, member templates *are* function templates. When we define a member template outside the body of a

class template, we must provide the template parameter list for the class template and for the function template. The parameter list for the class template comes first, followed by the member's own template parameter list:

```
template <typename T>          // type parameter for the class
template <typename It>         // type parameter for the constructor
    Blob<T>::Blob(It b, It e):
        data(std::make_shared<std::vector<T>>(b, e)) { }
```

Here we are defining a member of a class template that has one template type parameter, which we have named `T`. The member itself is a function template that has a type parameter named `It`.

## Instantiation and Member Templates

To instantiate a member template of a class template, we must supply arguments for the template parameters for both the class and the function templates. As usual, argument(s) for the class template parameter(s) are determined by the type of the object through which we call the member template. Also as usual, the compiler typically deduces template argument(s) for the member template's own parameter(s) from the arguments passed in the call (§ 16.1.1, p. 653):

```
int ia[] = {0,1,2,3,4,5,6,7,8,9};
vector<long> vi = {0,1,2,3,4,5,6,7,8,9};
list<const char*> w = {"now", "is", "the", "time"};

// instantiates the Blob<int> class
// and the Blob<int> constructor that has two int* parameters
Blob<int> a1(begin(ia), end(ia));
// instantiates the Blob<int> constructor that has
// two vector<long>::iterator parameters
Blob<int> a2(vi.begin(), vi.end());
// instantiates the Blob<string> class and the Blob<string>
// constructor that has two (list<const char*>::iterator parameters
Blob<string> a3(w.begin(), w.end());
```

When we define `a1`, we explicitly specify that the compiler should instantiate a version of `Blob` with the template parameter bound to `int`. The type parameter for the constructor's own parameters will be deduced from the type of `begin(ia)` and `end(ia)`. That type is `int*`. Thus, the definition of `a1` instantiates:

```
Blob<int>::Blob(int*, int*);
```

The definition of `a2` uses the already instantiated `Blob<int>` class, and instantiates the constructor with `It` replaced by `vector<short>::iterator`. The definition of `a3` (explicitly) instantiates the `Blob` with its template parameter bound to `string` and (implicitly) instantiates the member template constructor of that class with its parameter bound to `list<const char*>`.



**EXERCISES SECTION 16.1.4**

**Exercise 16.21:** Write your own version of `DebugDelete`.

**Exercise 16.22:** Revise your `TextQuery` programs from § 12.3 (p. 484) so that the `shared_ptr` members use a `DebugDelete` as their deleter (§ 12.1.4, p. 468).

**Exercise 16.23:** Predict when the call operator will be executed in your main query program. If your expectations and what happens differ, be sure you understand why.

**Exercise 16.24:** Add a constructor that takes two iterators to your `Blob` template.



## 16.1.5 Controlling Instantiations

The fact that instantiations are generated when a template is used (§ 16.1.1, p. 656) means that the same instantiation may appear in multiple object files. When two or more separately compiled source files use the same template with the same template arguments, there is an instantiation of that template in each of those files.

In large systems, the overhead of instantiating the same template in multiple files can become significant. Under the new standard, we can avoid this overhead through an **explicit instantiation**. An explicit instantiation has the form

C++  
11

```
extern template declaration; // instantiation declaration
template declaration;      // instantiation definition
```

where *declaration* is a class or function declaration in which all the template parameters are replaced by the template arguments. For example,

```
// instantiation declaration and definition
extern template class Blob<string>;           // declaration
template int compare(const int&, const int&); // definition
```

When the compiler sees an `extern template` declaration, it will not generate code for that instantiation in that file. Declaring an instantiation as `extern` is a promise that there will be a `nonextern` use of that instantiation elsewhere in the program. There may be several `extern` declarations for a given instantiation but there must be exactly one definition for that instantiation.

Because the compiler automatically instantiates a template when we use it, the `extern` declaration must appear before any code that uses that instantiation:

```
// Application.cc
// these template types must be instantiated elsewhere in the program
extern template class Blob<string>;
extern template int compare(const int&, const int&);
Blob<string> sa1, sa2; // instantiation will appear elsewhere
// Blob<int> and its initializer_list constructor instantiated in this file
Blob<int> a1 = {0,1,2,3,4,5,6,7,8,9};
Blob<int> a2(a1); // copy constructor instantiated in this file
int i = compare(a1[0], a2[0]); // instantiation will appear elsewhere
```

The file `Application.o` will contain instantiations for `Blob<int>`, along with the `initializer_list` and copy constructors for that class. The `compare<int>` function and `Blob<string>` class will not be instantiated in that file. There must be definitions of these templates in some other file in the program:

```
// templateBuild.cc
// instantiation file must provide a (nonextern) definition for every
// type and function that other files declare as extern
template int compare(const int&, const int&);
template class Blob<string>; // instantiates all members of the class template
```

When the compiler sees an instantiation definition (as opposed to a declaration), it generates code. Thus, the file `templateBuild.o` will contain the definitions for `compare` instantiated with `int` and for the `Blob<string>` class. When we build the application, we must link `templateBuild.o` with the `Application.o` files.



**WARNING**

There must be an explicit instantiation definition somewhere in the program for every instantiation declaration.

## Instantiation Definitions Instantiate All Members

An instantiation definition for a class template instantiates *all* the members of that template including inline member functions. When the compiler sees an instantiation definition it cannot know which member functions the program uses. Hence, unlike the way it handles ordinary class template instantiations, the compiler instantiates *all* the members of that class. Even if we do not use a member, that member will be instantiated. Consequently, we can use explicit instantiation only for types that can be used with all the members of that template.



An instantiation definition can be used only for types that can be used with every member function of a class template.



## 16.1.6 Efficiency and Flexibility

The library smart pointer types (§ 12.1, p. 450) offer a good illustration of design choices faced by designers of templates.

The obvious difference between `shared_ptr` and `unique_ptr` is the strategy they use in managing the pointer they hold—one class gives us shared ownership; the other owns the pointer that it holds. This difference is essential to what these classes do.

These classes also differ in how they let users override their default deleter. We can easily override the deleter of a `shared_ptr` by passing a callable object when we create or `reset` the pointer. In contrast, the type of the deleter is part of the type of a `unique_ptr` object. Users must supply that type as an explicit template argument when they define a `unique_ptr`. As a result, it is more complicated for users of `unique_ptr` to provide their own deleter.

**EXERCISES SECTION 16.1.5**

**Exercise 16.25:** Explain the meaning of these declarations:

```
extern template class vector<string>;
template class vector<Sales_data>;
```

**Exercise 16.26:** Assuming `NoDefault` is a class that does not have a default constructor, can we explicitly instantiate `vector<NoDefault>`? If not, why not?

**Exercise 16.27:** For each labeled statement explain what, if any, instantiations happen. If a template is instantiated, explain why; if not, explain why not.

```
template <typename T> class Stack { };
void f1(Stack<char>);                // (a)
class Exercise {
    Stack<double> &rsd;                // (b)
    Stack<int>    si;                  // (c)
};
int main() {
    Stack<char> *sc;                  // (d)
    f1(*sc);                         // (e)
    int iObj = sizeof(Stack< string >); // (f)
}
```

The difference in how the deleter is handled is incidental to the functionality of these classes. However, as we'll see, this difference in implementation strategy may have important performance impacts.

## Binding the Deleter at Run Time

Although we don't know how the library types are implemented, we can infer that `shared_ptr` must access its deleter indirectly. That is the deleter must be stored as a pointer or as a class (such as function (§ 14.8.3, p. 577)) that encapsulates a pointer.

We can be certain that `shared_ptr` does not hold the deleter as a direct member, because the type of the deleter isn't known until run time. Indeed, we can change the type of the deleter in a given `shared_ptr` during that `shared_ptr`'s lifetime. We can construct a `shared_ptr` using a deleter of one type, and subsequently use `reset` to give that same `shared_ptr` a different type of deleter. In general, we cannot have a member whose type changes at run time. Hence, the deleter must be stored indirectly.

To think about how the deleter must work, let's assume that `shared_ptr` stores the pointer it manages in a member named `p`, and that the deleter is accessed through a member named `del`. The `shared_ptr` destructor must include a statement such as

```
// value of del known only at run time; call through a pointer
del ? del(p) : delete p; // del(p) requires run-time jump to del's location
```

Because the deleter is stored indirectly, the call `del(p)` requires a run-time jump to the location stored in `del` to execute the code to which `del` points.

## Binding the Deleter at Compile Time

Now, let's think about how `unique_ptr` might work. In this class, the type of the deleter is part of the type of the `unique_ptr`. That is, `unique_ptr` has two template parameters, one that represents the pointer that the `unique_ptr` manages and the other that represents the type of the deleter. Because the type of the deleter is part of the type of a `unique_ptr`, the type of the deleter member is known at compile time. The deleter can be stored directly in each `unique_ptr` object.

The `unique_ptr` destructor operates similarly to its `shared_ptr` counterpart in that it calls a user-supplied deleter or executes `delete` on its stored pointer:

```
// del bound at compile time; direct call to the deleter is instantiated
del(p);    // no run-time overhead
```

The type of `del` is either the default deleter type or a user-supplied type. It doesn't matter; either way the code that will be executed is known at compile time. Indeed, if the deleter is something like our `DebugDelete` class (§ 16.1.4, p. 672) this call might even be inlined at compile time.

By binding the deleter at compile time, `unique_ptr` avoids the run-time cost of an indirect call to its deleter. By binding the deleter at run time, `shared_ptr` makes it easier for users to override the deleter.

### EXERCISES SECTION 16.1.6

**Exercise 16.28:** Write your own versions of `shared_ptr` and `unique_ptr`.

**Exercise 16.29:** Revise your `Blob` class to use your version of `shared_ptr` rather than the library version.

**Exercise 16.30:** Rerun some of your programs to verify your `shared_ptr` and revised `Blob` classes. (Note: Implementing the `weak_ptr` type is beyond the scope of this Primer, so you will not be able to use the `BlobPtr` class with your revised `Blob`.)

**Exercise 16.31:** Explain how the compiler might inline the call to the deleter if we used `DebugDelete` with `unique_ptr`.

## 16.2 Template Argument Deduction

We've seen that, by default, the compiler uses the arguments in a call to determine the template parameters for a function template. The process of determining the template arguments from the function arguments is known as **template argument deduction**. During template argument deduction, the compiler uses types of the arguments in the call to find the template arguments that generate a version of the function that best matches the given call.

## 16.2.1 Conversions and Template Type Parameters



As with a nontemplate function, the arguments we pass in a call to a function template are used to initialize that function's parameters. Function parameters whose type uses a template type parameter have special initialization rules. Only a very limited number of conversions are automatically applied to such arguments. Rather than converting the arguments, the compiler generates a new instantiation.

As usual, top-level `const`s (§ 2.4.3, p. 63) in either the parameter or the argument are ignored. The only other conversions performed in a call to a function template are

- **const conversions:** A function parameter that is a reference (or pointer) to a `const` can be passed a reference (or pointer) to a nonconst object (§ 4.11.2, p. 162).
- **Array- or function-to-pointer conversions:** If the function parameter is not a reference type, then the normal pointer conversion will be applied to arguments of array or function type. An array argument will be converted to a pointer to its first element. Similarly, a function argument will be converted to a pointer to the function's type (§ 4.11.2, p. 161).

Other conversions, such as the arithmetic conversions (§ 4.11.1, p. 159), derived-to-base (§ 15.2.2, p. 597), and user-defined conversions (§ 7.5.4, p. 294, and § 14.9, p. 579), are not performed.

As examples, consider calls to the functions `fobj` and `fref`. The `fobj` function copies its parameters, whereas `fref`'s parameters are references:

```
template <typename T> T fobj(T, T); // arguments are copied
template <typename T> T fref(const T&, const T&); // references

string s1("a value");
const string s2("another value");

fobj(s1, s2); // calls fobj(string, string); const is ignored
fref(s1, s2); // calls fref(const string&, const string&)
               // uses permissible conversion to const on s1

int a[10], b[42];
fobj(a, b); // calls f(int*, int*)
fref(a, b); // error: array types don't match
```

In the first pair of calls, we pass a `string` and a `const string`. Even though these types do not match exactly, both calls are legal. In the call to `fobj`, the arguments are copied, so whether the original object is `const` doesn't matter. In the call to `fref`, the parameter type is a reference to `const`. Conversion to `const` for a reference parameter is a permitted conversion, so this call is legal.

In the next pair of calls, we pass array arguments in which the arrays are different sizes and hence have different types. In the call to `fobj`, the fact that the array types differ doesn't matter. Both arrays are converted to pointers. The template parameter type in `fobj` is `int*`. The call to `fref`, however, is illegal. When the parameter is a reference, the arrays are not converted to pointers (§ 6.2.4, p. 217). The types of `a` and `b` don't match, so the call is in error.



`const` conversions and array or function to pointer are the only automatic conversions for arguments to parameters with template types.

## Function Parameters That Use the Same Template Parameter Type

A template type parameter can be used as the type of more than one function parameter. Because there are limited conversions, the arguments to such parameters must have essentially the same type. If the deduced types do not match, then the call is an error. For example, our `compare` function (§ 16.1.1, p. 652) takes two `const T&` parameters. Its arguments must have essentially the same type:

```
long lng;
compare(lng, 1024); // error: cannot instantiate compare(long, int)
```

This call is in error because the arguments to `compare` don't have the same type. The template argument deduced from the first argument is `long`; the one for the second is `int`. These types don't match, so template argument deduction fails.

If we want to allow normal conversions on the arguments, we can define the function with two type parameters:

```
// argument types can differ but must be compatible
template <typename A, typename B>
int flexibleCompare(const A& v1, const B& v2)
{
    if (v1 < v2) return -1;
    if (v2 < v1) return 1;
    return 0;
}
```

Now the user may supply arguments of different types:

```
long lng;
flexibleCompare(lng, 1024); // ok: calls flexibleCompare(long, int)
```

Of course, a `<` operator must exist that can compare values of those types.

## Normal Conversions Apply for Ordinary Arguments

A function template can have parameters that are defined using ordinary types—that is, types that do not involve a template type parameter. Such arguments have no special processing; they are converted as usual to the corresponding type of the parameter (§ 6.1, p. 203). For example, consider the following template:

```
template <typename T> ostream& print(ostream& os, const T& obj)
{
    return os << obj;
}
```

The first function parameter has a known type, `ostream&`. The second parameter, `obj`, has a template parameter type. Because the type of `os` is fixed, normal conversions are applied to arguments passed to `os` when `print` is called:

```
print(cout, 42); // instantiates print(ostream&, int)
ofstream f("output");
print(f, 10);    // uses print(ostream&, int); converts f to ostream&
```

In the first call, the type of the first argument exactly matches the type of the first parameter. This call will cause a version of `print` that takes an `ostream&` and an `int` to be instantiated. In the second call, the first argument is an `ofstream` and there is a conversion from `ofstream` to `ostream&` (§ 8.2.1, p. 317). Because the type of this parameter does not depend on a template parameter, the compiler will implicitly convert `f` to `ostream&`.



Normal conversions are applied to arguments whose type is not a template parameter.

## EXERCISES SECTION 16.2.1

**Exercise 16.32:** What happens during template argument deduction?

**Exercise 16.33:** Name two type conversions allowed on function arguments involved in template argument deduction.

**Exercise 16.34:** Given only the following code, explain whether each of these calls is legal. If so, what is the type of `T`? If not, why not?

```
template <class T> int compare(const T&, const T&);
(a) compare("hi", "world"); (b) compare("bye", "dad");
```

**Exercise 16.35:** Which, if any, of the following calls are errors? If the call is legal, what is the type of `T`? If the call is not legal, what is the problem?

```
template <typename T> T calc(T, int);
template <typename T> T fcn(T, T);
double d; float f; char c;
(a) calc(c, 'c'); (b) calc(d, f);
(c) fcn(c, 'c'); (d) fcn(d, f);
```

**Exercise 16.36:** What happens in the following calls:

```
template <typename T> f1(T, T);
template <typename T1, typename T2> f2(T1, T2);
int i = 0, j = 42, *p1 = &i, *p2 = &j;
const int *cp1 = &i, *cp2 = &j;
(a) f1(p1, p2); (b) f2(p1, p2); (c) f1(cp1, cp2);
(d) f2(cp1, cp2); (e) f1(p1, cp1); (f) f2(p1, cp1);
```

## 16.2.2 Function-Template Explicit Arguments



In some situations, it is not possible for the compiler to deduce the types of the template arguments. In others, we want to allow the user to control the template

instantiation. Both cases arise most often when a function return type differs from any of those used in the parameter list.

## Specifying an Explicit Template Argument

As an example in which we want to let the user specify which type to use, we'll define a function template named `sum` that takes arguments of two different types. We'd like to let the user specify the type of the result. That way the user can choose whatever precision is appropriate.

We can let the user control the type of the return by defining a third template parameter to represent the return type:

```
// T1 cannot be deduced: it doesn't appear in the function parameter list
template <typename T1, typename T2, typename T3>
T1 sum(T2, T3);
```

In this case, there is no argument whose type can be used to deduce the type of `T1`. The caller must provide an **explicit template argument** for this parameter on each call to `sum`.

We supply an explicit template argument to a call the same way that we define an instance of a class template. Explicit template arguments are specified inside angle brackets after the function name and before the argument list:

```
// T1 is explicitly specified; T2 and T3 are inferred from the argument types
auto val3 = sum<long long>(i, lng); // long long sum(int, long)
```

This call explicitly specifies the type for `T1`. The compiler will deduce the types for `T2` and `T3` from the types of `i` and `lng`.

Explicit template argument(s) are matched to corresponding template parameter(s) from left to right; the first template argument is matched to the first template parameter, the second argument to the second parameter, and so on. An explicit template argument may be omitted only for the trailing (right-most) parameters, and then only if these can be deduced from the function parameters. If our `sum` function had been written as

```
// poor design: users must explicitly specify all three template parameters
template <typename T1, typename T2, typename T3>
T3 alternative_sum(T2, T1);
```

then we would always have to specify arguments for all three parameters:

```
// error: can't infer initial template parameters
auto val3 = alternative_sum<long long>(i, lng);

// ok: all three parameters are explicitly specified
auto val2 = alternative_sum<long long, int, long>(i, lng);
```

## Normal Conversions Apply for Explicitly Specified Arguments

For the same reasons that normal conversions are permitted for parameters that are defined using ordinary types (§ 16.2.1, p. 680), normal conversions also apply for arguments whose template type parameter is explicitly specified:



```

long lng;
compare(lng, 1024);           // error: template parameters don't match
compare<long>(lng, 1024);     // ok: instantiates compare(long, long)
compare<int>(lng, 1024);      // ok: instantiates compare(int, int)

```

As we've seen, the first call is in error because the arguments to `compare` must have the same type. If we explicitly specify the template parameter type, normal conversions apply. Thus, the call to `compare<long>` is equivalent to calling a function taking two `const long&` parameters. The `int` parameter is automatically converted to `long`. In the second call, `T` is explicitly specified as `int`, so `lng` is converted to `int`.

## EXERCISES SECTION 16.2.2

**Exercise 16.37:** The library `max` function has two function parameters and returns the larger of its arguments. This function has one template type parameter. Could you call `max` passing it an `int` and a `double`? If so, how? If not, why not?

**Exercise 16.38:** When we call `make_shared` (§ 12.1.1, p. 451), we have to provide an explicit template argument. Explain why that argument is needed and how it is used.

**Exercise 16.39:** Use an explicit template argument to make it sensible to pass two string literals to the original version of `compare` from § 16.1.1 (p. 652).

## 16.2.3 Trailing Return Types and Type Transformation



Using an explicit template argument to represent a template function's return type works well when we want to let the user determine the return type. In other cases, requiring an explicit template argument imposes a burden on the user with no compensating advantage. For example, we might want to write a function that takes a pair of iterators denoting a sequence and returns a reference to an element in the sequence:

```

template <typename It>
??? &fcn(It beg, It end)
{
    // process the range
    return *beg; // return a reference to an element from the range
}

```

We don't know the exact type we want to return, but we do know that we want that type to be a reference to the element type of the sequence we're processing:

```

vector<int> vi = {1,2,3,4,5};
Blob<string> ca = { "hi", "bye" };
auto &i = fcn(vi.begin(), vi.end()); // fcn should return int&
auto &s = fcn(ca.begin(), ca.end()); // fcn should return string&

```

Here, we know that our function will return `*beg`, and we know that we can use `decltype(*beg)` to obtain the type of that expression. However, `beg` doesn't exist until the parameter list has been seen. To define this function, we must use a trailing return type (§ 6.3.3, p. 229). Because a trailing return appears after the parameter list, it can use the function's parameters:

**C++  
11**

```
// a trailing return lets us declare the return type after the parameter list is seen
template <typename It>
auto fcn(It beg, It end) -> decltype(*beg)
{
    // process the range
    return *beg;    // return a reference to an element from the range
}
```

Here we've told the compiler that `fcn`'s return type is the same as the type returned by dereferencing its `beg` parameter. The dereference operator returns an lvalue (§ 4.1.1, p. 136), so the type deduced by `decltype` is a reference to the type of the element that `beg` denotes. Thus, if `fcn` is called on a sequence of strings, the return type will be `string&`. If the sequence is `int`, the return will be `int&`.

## The Type Transformation Library Template Classes

Sometimes we do not have direct access to the type that we need. For example, we might want to write a function similar to `fcn` that returns an element by value (§ 6.3.2, p. 224), rather than a reference to an element.

The problem we face in writing this function is that we know almost nothing about the types we're passed. In this function, the only operations we know we can use are iterator operations, and there are no iterator operations that yield elements (as opposed to references to elements).

To obtain the element type, we can use a library **type transformation** template. These templates are defined in the `type_traits` header. In general the classes in `type_traits` are used for so-called template metaprogramming, a topic that is beyond the scope of this Primer. However, the type transformation templates are useful in ordinary programming as well. These templates are described in Table 16.1 and we'll see how they are implemented in § 16.5 (p. 710).

In this case, we can use `remove_reference` to obtain the element type. The `remove_reference` template has one template type parameter and a (public) type member named `type`. If we instantiate `remove_reference` with a reference type, then `type` will be the referred-to type. For example, if we instantiate `remove_reference<int&>`, the `type` member will be `int`. Similarly, if we instantiate `remove_reference<string&>`, `type` will be `string`, and so on. More generally, given that `beg` is an iterator:

```
remove_reference<decltype(*beg)>::type
```

will be the type of the element to which `beg` refers: `decltype(*beg)` returns the reference type of the element type. `remove_reference::type` strips off the reference, leaving the element type itself.

Using `remove_reference` and a trailing return with `decltype`, we can write our function to return a copy of an element's value:

```
// must use typename to use a type member of a template parameter; see § 16.1.3 (p. 670)
template <typename It>
auto fcn2(It beg, It end) ->
    typename remove_reference<decltype(*beg)>::type
{
    // process the range
    return *beg; // return a copy of an element from the range
}
```

Note that `type` is member of a class that depends on a template parameter. As a result, we must use `typename` in the declaration of the return type to tell the compiler that `type` represents a type (§ 16.1.3, p. 670).

| Table 16.1: Standard Type Transformation Templates        |                                                                       |                                                              |
|-----------------------------------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------|
| For <code>Mod&lt;T&gt;</code> , where <code>Mod</code> is | If <code>T</code> is                                                  | Then <code>Mod&lt;T&gt;::type</code> is                      |
| <code>remove_reference</code>                             | <code>X&amp;</code> or <code>X&amp;&amp;</code><br>otherwise          | <code>X</code><br><code>T</code>                             |
| <code>add_const</code>                                    | <code>X&amp;</code> , <code>const X</code> , or function<br>otherwise | <code>T</code><br><code>const T</code>                       |
| <code>add_lvalue_reference</code>                         | <code>X&amp;</code><br><code>X&amp;&amp;</code><br>otherwise          | <code>T</code><br><code>X&amp;</code><br><code>T&amp;</code> |
| <code>add_rvalue_reference</code>                         | <code>X&amp;</code> or <code>X&amp;&amp;</code><br>otherwise          | <code>T</code><br><code>T&amp;&amp;</code>                   |
| <code>remove_pointer</code>                               | <code>X*</code><br>otherwise                                          | <code>X</code><br><code>T</code>                             |
| <code>add_pointer</code>                                  | <code>X&amp;</code> or <code>X&amp;&amp;</code><br>otherwise          | <code>X*</code><br><code>T*</code>                           |
| <code>make_signed</code>                                  | <code>unsigned X</code><br>otherwise                                  | <code>X</code><br><code>T</code>                             |
| <code>make_unsigned</code>                                | <code>signed type</code><br>otherwise                                 | <code>unsigned T</code><br><code>T</code>                    |
| <code>remove_extent</code>                                | <code>X[n]</code><br>otherwise                                        | <code>X</code><br><code>T</code>                             |
| <code>remove_all_extents</code>                           | <code>X[n1][n2]...</code><br>otherwise                                | <code>X</code><br><code>T</code>                             |

Each of the type transformation templates described in Table 16.1 works similarly to `remove_reference`. Each template has a public member named `type` that represents a type. That type may be related to the template’s own template type parameter in a way that is indicated by the template’s name. If it is not possible (or not necessary) to transform the template’s parameter, the `type` member is the template parameter type itself. For example, if `T` is a pointer type, then `remove_pointer<T>::type` is the type to which `T` points. If `T` isn’t a pointer, then no transformation is needed. In this case, `type` is the same type as `T`.

**EXERCISES SECTION 16.2.3**

**Exercise 16.40:** Is the following function legal? If not, why not? If it is legal, what, if any, are the restrictions on the argument type(s) that can be passed, and what is the return type?

```
template <typename It>
auto fcn3(It beg, It end) -> decltype(*beg + 0)
{
    // process the range
    return *beg; // return a copy of an element from the range
}
```

**Exercise 16.41:** Write a version of `sum` with a return type that is guaranteed to be large enough to hold the result of the addition.



## 16.2.4 Function Pointers and Argument Deduction

When we initialize or assign a function pointer (§ 6.7, p. 247) from a function template, the compiler uses the type of the pointer to deduce the template argument(s).

As an example, assume we have a function pointer that points to a function returning an `int` that takes two parameters, each of which is a reference to a `const int`. We can use that pointer to point to an instantiation of `compare`:

```
template <typename T> int compare(const T&, const T&);
// pfl points to the instantiation int compare(const int&, const int&)
int (*pfl)(const int&, const int&) = compare;
```

The type of the parameters in `pfl` determines the type of the template argument for `T`. The template argument for `T` is `int`. The pointer `pfl` points to the instantiation of `compare` with `T` bound to `int`. It is an error if the template arguments cannot be determined from the function pointer type:

```
// overloaded versions of func; each takes a different function pointer type
void func(int (*)(const string&, const string&));
void func(int (*)(const int&, const int&));
func(compare); // error: which instantiation of compare?
```

The problem is that by looking at the type of `func`'s parameter, it is not possible to determine a unique type for the template argument. The call to `func` could instantiate the version of `compare` that takes `ints` or the version that takes `strings`. Because it is not possible to identify a unique instantiation for the argument to `func`, this call won't compile.

We can disambiguate the call to `func` by using explicit template arguments:

```
// ok: explicitly specify which version of compare to instantiate
func(compare<int>); // passing compare(const int&, const int&)
```

This expression calls the version of `func` that takes a function pointer with two `const int&` parameters.



When the address of a function-template instantiation is taken, the context must be such that it allows a unique type or value to be determined for each template parameter.

## 16.2.5 Template Argument Deduction and References



In order to understand type deduction from a call to a function such as

```
template <typename T> void f(T &p);
```

in which the function's parameter `p` is a reference to a template type parameter `T`, it is important to keep in mind two points: Normal reference binding rules apply; and `const`s are low level, not top level.

### Type Deduction from Lvalue Reference Function Parameters

When a function parameter is an ordinary (lvalue) reference to a template type parameter (i.e., that has the form `T&`), the binding rules say that we can pass only an lvalue (e.g., a variable or an expression that returns a reference type). That argument might or might not have a `const` type. If the argument is `const`, then `T` will be deduced as a `const` type:

```
template <typename T> void f1(T&); // argument must be an lvalue
// calls to f1 use the referred-to type of the argument as the template parameter type
f1(i); // i is an int; template parameter T is int
f1(ci); // ci is a const int; template parameter T is const int
f1(5); // error: argument to a & parameter must be an lvalue
```

If a function parameter has type `const T&`, normal binding rules say that we can pass any kind of argument—an object (`const` or otherwise), a temporary, or a literal value. When the function parameter is itself `const`, the type deduced for `T` will not be a `const` type. The `const` is already part of the *function* parameter type; therefore, it does not also become part of the *template* parameter type:

```
template <typename T> void f2(const T&); // can take an rvalue
// parameter in f2 is const &; const in the argument is irrelevant
// in each of these three calls, f2's function parameter is inferred as const int&
f2(i); // i is an int; template parameter T is int
f2(ci); // ci is a const int, but template parameter T is int
f2(5); // a const & parameter can be bound to an rvalue; T is int
```

### Type Deduction from Rvalue Reference Function Parameters

When a function parameter is an rvalue reference (§ 13.6.1, p. 532) (i.e., has the form `T&&`), normal binding rules say that we can pass an rvalue to this parameter. When we do so, type deduction behaves similarly to deduction for an ordinary lvalue reference function parameter. The deduced type for `T` is the type of the rvalue:

```
template <typename T> void f3(T&&);
f3(42); // argument is an rvalue of type int; template parameter T is int
```

## Reference Collapsing and Rvalue Reference Parameters

Assuming `i` is an `int` object, we might think that a call such as `f3(i)` would be illegal. After all, `i` is an lvalue, and normally we cannot bind an rvalue reference to an lvalue. However, the language defines two exceptions to normal binding rules that allow this kind of usage. These exceptions are the foundation for how library facilities such as `move` operate.

The first exception affects how type deduction is done for rvalue reference parameters. When we pass an lvalue (e.g., `i`) to a function parameter that is an rvalue reference to a template type parameter (e.g., `T&&`), the compiler deduces the template type parameter as the argument's lvalue reference type. So, when we call `f3(i)`, the compiler deduces the type of `T` as `int&`, not `int`.

Deducing `T` as `int&` would seem to mean that `f3`'s function parameter would be an rvalue reference to the type `int&`. Ordinarily, we cannot (directly) define a reference to a reference (§ 2.3.1, p. 51). However, it is possible to do so indirectly through a type alias (§ 2.5.1, p. 67) or through a template type parameter.

In such contexts, we see the second exception to the normal binding rules: If we indirectly create a reference to a reference, then those references “collapse.” In all but one case, the references collapse to form an ordinary lvalue reference type. The new standard, expanded the collapsing rules to include rvalue references. References collapse to form an rvalue reference only in the specific case of an rvalue reference to an rvalue reference. That is, for a given type `X`:

- `X& &`, `X& &&`, and `X&& &` all collapse to type `X&`
- The type `X&& &&` collapses to `X&&`

C++  
11



Reference collapsing applies only when a reference to a reference is created indirectly, such as in a type alias or a template parameter.

The combination of the reference collapsing rule and the special rule for type deduction for rvalue reference parameters means that we can call `f3` on an lvalue. When we pass an lvalue to `f3`'s (rvalue reference) function parameter, the compiler will deduce `T` as an lvalue reference type:

```
f3(i); // argument is an lvalue; template parameter T is int&
f3(ci); // argument is an lvalue; template parameter T is const int&
```

When a *template* parameter `T` is deduced as a reference type, the collapsing rule says that the *function* parameter `T&&` collapses to an lvalue reference type. For example, the resulting instantiation for `f3(i)` would be something like

```
// invalid code, for illustration purposes only
void f3<int&>(int& &&); // when T is int&, function parameter is int& &&
```

The function parameter in `f3` is `T&&` and `T` is `int&`, so `T&&` is `int& &&`, which collapses to `int&`. Thus, even though the form of the function parameter in `f3` is an rvalue reference (i.e., `T&&`), this call instantiates `f3` with an lvalue reference type (i.e., `int&`):

```
void f3<int&>(int&); // when T is int&, function parameter collapses to int&
```

There are two important consequences from these rules:

- A function parameter that is an rvalue reference to a template type parameter (e.g., `T&&`) can be bound to an lvalue; and
- If the argument is an lvalue, then the deduced template argument type will be an lvalue reference type and the function parameter will be instantiated as an (ordinary) lvalue reference parameter (`T&`)

It is also worth noting that by implication, we can pass any type of argument to a `T&&` function parameter. A parameter of such a type can (obviously) be used with rvalues, and as we've just seen, can be used by lvalues as well.



An argument of any type can be passed to a function parameter that is an rvalue reference to a template parameter type (i.e., `T&&`). When an lvalue is passed to such a parameter, the function parameter is instantiated as an ordinary, lvalue reference (`T&`).

## Writing Template Functions with Rvalue Reference Parameters

The fact that the template parameter can be deduced to a reference type can have surprising impacts on the code inside the template:

```
template <typename T> void f3(T&& val)
{
    T t = val;    // copy or binding a reference?
    t = fcn(t);   // does the assignment change only t or val and t?
    if (val == t) { /* ... */ } // always true if T is a reference type
}
```

When we call `f3` on an rvalue, such as the literal `42`, `T` is `int`. In this case, the local variable `t` has type `int` and is initialized by copying the value of the parameter `val`. When we assign to `t`, the parameter `val` remains unchanged.

On the other hand, when we call `f3` on the lvalue `i`, then `T` is `int&`. When we define and initialize the local variable `t`, that variable has type `int&`. The initialization of `t` binds `t` to `val`. When we assign to `t`, we change `val` at the same time. In this instantiation of `f3`, the `if` test will always yield `true`.

It is surprisingly hard to write code that is correct when the types involved might be plain (nonreference) types or reference types (although the type transformation classes such as `remove_reference` can help (§ 16.2.3, p. 684)).

In practice, rvalue reference parameters are used in one of two contexts: Either the template is forwarding its arguments, or the template is overloaded. We'll look at forwarding in § 16.2.7 (p. 692) and at template overloading in § 16.3 (p. 694).

For now, it's worth noting that function templates that use rvalue references often use overloading in the same way as we saw in § 13.6.3 (p. 544):

```
template <typename T> void f(T&&);           // binds to nonconst rvalues
template <typename T> void f(const T&);     // lvalues and const rvalues
```

As with nontemplate functions, the first version will bind to modifiable rvalues and the second to lvalues or to `const` rvalues.

**EXERCISES SECTION 16.2.5**

**Exercise 16.42:** Determine the type of `T` and of `val` in each of the following calls:

```
template <typename T> void g(T&& val);
int i = 0; const int ci = i;
(a) g(i);           (b) g(ci);           (c) g(i * ci);
```

**Exercise 16.43:** Using the function defined in the previous exercise, what would the template parameter of `g` be if we called `g(i = ci)`?

**Exercise 16.44:** Using the same three calls as in the first exercise, determine the types for `T` if `g`'s function parameter is declared as `T` (not `T&&`). What if `g`'s function parameter is `const T&`?

**Exercise 16.45:** Given the following template, explain what happens if we call `g` on a literal value such as `42`. What if we call `g` on a variable of type `int`?

```
template <typename T> void g(T&& val) { vector<T> v; }
```



## 16.2.6 Understanding `std::move`

The library `move` function (§ 13.6.1, p. 533) is a good illustration of a template that uses rvalue references. Fortunately, we can use `move` without understanding the template mechanisms that it uses. However, looking at how `move` works can help cement our general understanding, and use, of templates.

In § 13.6.2 (p. 534) we noted that although we cannot directly bind an rvalue reference to an lvalue, we can use `move` to obtain an rvalue reference bound to an lvalue. Because `move` can take arguments of essentially any type, it should not be surprising that `move` is a function template.

### How `std::move` Is Defined

The standard defines `move` as follows:

```
// for the use of typename in the return type and the cast see § 16.1.3 (p. 670)
// remove_reference is covered in § 16.2.3 (p. 684)
template <typename T>
typename remove_reference<T>::type&& move(T&& t)
{
    // static_cast covered in § 4.11.3 (p. 163)
    return static_cast<typename remove_reference<T>::type&&>(t);
}
```

This code is short but subtle. First, `move`'s function parameter, `T&&`, is an rvalue reference to a template parameter type. Through reference collapsing, this parameter can match arguments of any type. In particular, we can pass either an lvalue or an rvalue to `move`:

```
string s1("hi!"), s2;
s2 = std::move(string("bye!")); // ok: moving from an rvalue
s2 = std::move(s1);           // ok: but after the assignment s1 has indeterminate value
```



## How `std::move` Works



In the first assignment, the argument to `move` is the rvalue result of the `string` constructor, `string("bye")`. As we've seen, when we pass an rvalue to an rvalue reference function parameter, the type deduced from that argument is the referred-to type (§ 16.2.5, p. 687). Thus, in `std::move(string("bye!"))`:

- The deduced type of `T` is `string`.
- Therefore, `remove_reference` is instantiated with `string`.
- The type member of `remove_reference<string>` is `string`.
- The return type of `move` is `string&&`.
- `move`'s function parameter, `t`, has type `string&&`.

Accordingly, this call instantiates `move<string>`, which is the function

```
string&& move(string &&t)
```

The body of this function returns `static_cast<string&&>(t)`. The type of `t` is already `string&&`, so the cast does nothing. Therefore, the result of this call is the rvalue reference it was given.

Now consider the second assignment, which calls `std::move(s1)`. In this call, the argument to `move` is an lvalue. This time:

- The deduced type of `T` is `string&` (reference to `string`, not plain `string`).
- Therefore, `remove_reference` is instantiated with `string&`.
- The type member of `remove_reference<string&>` is `string`,
- The return type of `move` is still `string&&`.
- `move`'s function parameter, `t`, instantiates as `string& &&`, which collapses to `string&`.

Thus, this call instantiates `move<string&>`, which is

```
string&& move(string &t)
```

and which is exactly what we're after—we want to bind an rvalue reference to an lvalue. The body of this instantiation returns `static_cast<string&&>(t)`. In this case, the type of `t` is `string&`, which the cast converts to `string&&`.

## `static_cast` from an Lvalue to an Rvalue Reference Is Permitted

Ordinarily, a `static_cast` can perform only otherwise legitimate conversions (§ 4.11.3, p. 163). However, there is again a special dispensation for rvalue references: Even though we cannot implicitly convert an lvalue to an rvalue reference, we can *explicitly* cast an lvalue to an rvalue reference using `static_cast`.

C++  
11

Binding an rvalue reference to an lvalue gives code that operates on the rvalue reference permission to clobber the lvalue. There are times, such as in our `StrVec`

reallocate function in § 13.6.1 (p. 533), when we know it is safe to clobber an lvalue. By *letting* us do the cast, the language allows this usage. By *forcing* us to use a cast, the language tries to prevent us from doing so accidentally.

Finally, although we can write such casts directly, it is much easier to use the library move function. Moreover, using `std::move` consistently makes it easy to find the places in our code that might potentially clobber lvalues.

## EXERCISES SECTION 16.2.6

**Exercise 16.46:** Explain this loop from `StrVec::reallocate` in § 13.5 (p. 530):

```
for (size_t i = 0; i != size(); ++i)
    alloc.construct(dest++, std::move(*elem++));
```



## 16.2.7 Forwarding

Some functions need to forward one or more of their arguments with their types *unchanged* to another, forwarded-to, function. In such cases, we need to preserve everything about the forwarded arguments, including whether or not the argument type is `const`, and whether the argument is an lvalue or an rvalue.

As an example, we'll write a function that takes a callable expression and two additional arguments. Our function will call the given callable with the other two arguments in reverse order. The following is a first cut at our flip function:

```
// template that takes a callable and two parameters
// and calls the given callable with the parameters "flipped"
// flip1 is an incomplete implementation: top-level const and references are lost
template <typename F, typename T1, typename T2>
void flip1(F f, T1 t1, T2 t2)
{
    f(t2, t1);
}
```

This template works fine until we want to use it to call a function that has a reference parameter:

```
void f(int v1, int &v2) // note v2 is a reference
{
    cout << v1 << " " << ++v2 << endl;
}
```

Here `f` changes the value of the argument bound to `v2`. However, if we call `f` through `flip1`, the changes made by `f` do not affect the original argument:

```
f(42, i); // f changes its argument i
flip1(f, j, 42); // f called through flip1 leaves j unchanged
```

The problem is that `j` is passed to the `t1` parameter in `flip1`. That parameter has a plain, nonreference type, `int`, not an `int&`. That is, the instantiation of this call to `flip1` is

```
void flip1(void(*fcn)(int, int&), int t1, int t2);
```

The value of `j` is copied into `t1`. The reference parameter in `f` is bound to `t1`, not to `j`.

## Defining Function Parameters That Retain Type Information



To pass a reference through our `flip` function, we need to rewrite our function so that its parameters preserve the “lvalueness” of its given arguments. Thinking ahead a bit, we can imagine that we’d also like to preserve the `const`ness of the arguments as well.

We can preserve all the type information in an argument by defining its corresponding function parameter as an rvalue reference to a template type parameter. Using a reference parameter (either lvalue or rvalue) lets us preserve `const`ness, because the `const` in a reference type is low-level. Through reference collapsing (§ 16.2.5, p. 688), if we define the function parameters as `T1&&` and `T2&&`, we can preserve the lvalue/rvalue property of `flip`’s arguments (§ 16.2.5, p. 687):

```
template <typename F, typename T1, typename T2>
void flip2(F f, T1 &&t1, T2 &&t2)
{
    f(t2, t1);
}
```

As in our earlier call, if we call `flip2(f, j, 42)`, the lvalue `j` is passed to the parameter `t1`. However, in `flip2`, the type deduced for `T1` is `int&`, which means that the type of `t1` collapses to `int&`. The reference `t1` is bound to `j`. When `flip2` calls `f`, the reference parameter `v2` in `f` is bound to `t1`, which in turn is bound to `j`. When `f` increments `v2`, it is changing the value of `j`.



A function parameter that is an rvalue reference to a template type parameter (i.e., `T&&`) preserves the `const`ness and lvalue/rvalue property of its corresponding argument.

This version of `flip2` solves one half of our problem. Our `flip2` function works fine for functions that take lvalue references but cannot be used to call a function that has an rvalue reference parameter. For example:

```
void g(int &&i, int& j)
{
    cout << i << " " << j << endl;
}
```

If we try to call `g` through `flip2`, we will be passing the parameter `t2` to `g`’s rvalue reference parameter. Even if we pass an rvalue to `flip2`:

```
flip2(g, i, 42); // error: can't initialize int&& from an lvalue
```

what is passed to `g` will be the parameter named `t2` inside `flip2`. A function parameter, like any other variable, is an lvalue expression (§ 13.6.1, p. 533). As a result, the call to `g` in `flip2` passes an lvalue to `g`’s rvalue reference parameter.



## Using `std::forward` to Preserve Type Information in a Call

C++  
11

We can use a new library facility named `forward` to pass `flip2`'s parameters in a way that preserves the types of the original arguments. Like `move`, `forward` is defined in the utility header. Unlike `move`, `forward` must be called with an explicit template argument (§ 16.2.2, p. 682). `forward` returns an rvalue reference to that explicit argument type. That is, the return type of `forward<T>` is `T&&`.

Ordinarily, we use `forward` to pass a function parameter that is defined as an rvalue reference to a template type parameter. Through reference collapsing on its return type, `forward` preserves the lvalue/rvalue nature of its given argument:

```
template <typename Type> intermediary(Type &&arg)
{
    finalFcn(std::forward<Type>(arg));
    // ...
}
```

Here we use `Type`—which is deduced from `arg`—as `forward`'s explicit template argument type. Because `arg` is an rvalue reference to a template type parameter, `Type` will represent all the type information in the argument passed to `arg`. If that argument was an rvalue, then `Type` is an ordinary (nonreference) type and `forward<Type>` will return `Type&&`. If the argument was an lvalue, then—through reference collapsing—`Type` itself is an lvalue reference type. In this case, the return type is an rvalue reference to an lvalue reference type. Again through reference collapsing—this time on the return type—`forward<Type>` will return an lvalue reference type.



When used with a function parameter that is an rvalue reference to template type parameter (`T&&`), `forward` preserves all the details about an argument's type.

Using `forward`, we'll rewrite our `flip` function once more:

```
template <typename F, typename T1, typename T2>
void flip(F f, T1 &&t1, T2 &&t2)
{
    f(std::forward<T2>(t2), std::forward<T1>(t1));
}
```

If we call `flip(g, i, 42)`, `i` will be passed to `g` as an `int&` and `42` will be passed as an `int&&`.



As with `std::move`, it's a good idea not to provide a `using` declaration for `std::forward`. § 18.2.3 (p. 798) will explain why.



## 16.3 Overloading and Templates

Function templates can be overloaded by other templates or by ordinary, nontemplate functions. As usual, functions with the same name must differ either as to the number or the type(s) of their parameters.

**EXERCISES SECTION 16.2.7**

**Exercise 16.47:** Write your own version of the flip function and test it by calling functions that have lvalue and rvalue reference parameters.

Function matching (§ 6.4, p. 233) is affected by the presence of function templates in the following ways:

- The candidate functions for a call include any function-template instantiation for which template argument deduction (§ 16.2, p. 678) succeeds.
- The candidate function templates are always viable, because template argument deduction will have eliminated any templates that are not viable.
- As usual, the viable functions (template and nontemplate) are ranked by the conversions, if any, needed to make the call. Of course, the conversions used to call a function template are quite limited (§ 16.2.1, p. 679).
- Also as usual, if exactly one function provides a better match than any of the others, that function is selected. However, if there are several functions that provide an equally good match, then:
  - If there is only one nontemplate function in the set of equally good matches, the nontemplate function is called.
  - If there are no nontemplate functions in the set, but there are multiple function templates, and one of these templates is more specialized than any of the others, the more specialized function template is called.
  - Otherwise, the call is ambiguous.

**WARNING**

Correctly defining a set of overloaded function templates requires a good understanding of the relationship among types and of the restricted conversions applied to arguments in template functions.

**Writing Overloaded Templates**

As an example, we'll build a set of functions that might be useful during debugging. We'll name our debugging functions `debug_rep`, each of which will return a string representation of a given object. We'll start by writing the most general version of this function as a template that takes a reference to a `const` object:

```
// print any type we don't otherwise handle
template <typename T> string debug_rep(const T &t)
{
    ostringstream ret; // see § 8.3 (p. 321)
    ret << t; // uses T's output operator to print a representation of t
    return ret.str(); // return a copy of the string to which ret is bound
}
```

This function can be used to generate a string corresponding to an object of any type that has an output operator.

Next, we'll define a version of `debug_rep` to print pointers:

```
// print pointers as their pointer value, followed by the object to which the pointer points
// NB: this function will not work properly with char*; see § 16.3 (p. 698)
template <typename T> string debug_rep(T *p)
{
    ostreamstream ret;
    ret << "pointer: " << p;           // print the pointer's own value
    if (p)
        ret << " " << debug_rep(*p); // print the value to which p points
    else
        ret << " null pointer";       // or indicate that the p is null
    return ret.str(); // return a copy of the string to which ret is bound
}
```

This version generates a string that contains the pointer's own value and calls `debug_rep` to print the object to which that pointer points. Note that this function can't be used to print character pointers, because the IO library defines a version of the `<<` for `char*` values. That version of `<<` assumes the pointer denotes a null-terminated character array, and prints the contents of the array, not its address. We'll see in § 16.3 (p. 698) how to handle character pointers.

We might use these functions as follows:

```
string s("hi");
cout << debug_rep(s) << endl;
```

For this call, only the first version of `debug_rep` is viable. The second version of `debug_rep` requires a pointer parameter, and in this call we passed a nonpointer object. There is no way to instantiate a function template that expects a pointer type from a nonpointer argument, so argument deduction fails. Because there is only one viable function, that is the one that is called.

If we call `debug_rep` with a pointer:

```
cout << debug_rep(&s) << endl;
```

both functions generate viable instantiations:

- `debug_rep(const string*&)`, which is the instantiation of the first version of `debug_rep` with `T` bound to `string*`
- `debug_rep(string*)`, which is the instantiation of the second version of `debug_rep` with `T` bound to `string`

The instantiation of the second version of `debug_rep` is an exact match for this call. The instantiation of the first version requires a conversion of the plain pointer to a pointer to `const`. Normal function matching says we should prefer the second template, and indeed that is the one that is run.

## Multiple Viable Templates

As another example, consider the following call:

```
const string *sp = &s;
cout << debug_rep(sp) << endl;
```

Here both templates are viable and both provide an exact match:

- `debug_rep(const string&)`, the instantiation of the first version of the template with `T` bound to `const string*`
- `debug_rep(const string*)`, the instantiation of the second version of the template with `T` bound to `const string`

In this case, normal function matching can't distinguish between these two calls. We might expect this call to be ambiguous. However, due to the special rule for overloaded function templates, this call resolves to `debug_rep(T*)`, which is the more specialized template.

The reason for this rule is that without it, there would be no way to call the pointer version of `debug_rep` on a pointer to `const`. The problem is that the template `debug_rep(const T&)` can be called on essentially any type, including pointer types. That template is more general than `debug_rep(T*)`, which can be called only on pointer types. Without this rule, calls that passed pointers to `const` would always be ambiguous.



When there are several overloaded templates that provide an equally good match for a call, the most specialized version is preferred.

## Nontemplate and Template Overloads

For our next example, we'll define an ordinary nontemplate version of `debug_rep` to print strings inside double quotes:

```
// print strings inside double quotes
string debug_rep(const string &s)
{
    return '"' + s + '"';
}
```

Now, when we call `debug_rep` on a string,

```
string s("hi");
cout << debug_rep(s) << endl;
```

there are two equally good viable functions:

- `debug_rep<string>(const string&)`, the first template with `T` bound to `string`
- `debug_rep(const string&)`, the ordinary, nontemplate function

In this case, both functions have the same parameter list, so obviously, each function provides an equally good match for this call. However, the nontemplate version is selected. For the same reasons that the most specialized of equally good function templates is preferred, a nontemplate function is preferred over equally good match(es) to a function template.



When a nontemplate function provides an equally good match for a call as a function template, the nontemplate version is preferred.

## Overloaded Templates and Conversions

There's one case we haven't covered so far: pointers to C-style character strings and string literals. Now that we have a version of `debug_rep` that takes a `string`, we might expect that a call that passes character strings would match that version. However, consider this call:

```
cout << debug_rep("hi world!") << endl; // calls debug_rep(T*)
```

Here all three of the `debug_rep` functions are viable:

- `debug_rep(const T&)`, with `T` bound to `char[10]`
- `debug_rep(T*)`, with `T` bound to `const char`
- `debug_rep(const string&)`, which requires a conversion from `const char*` to `string`

Both templates provide an exact match to the argument—the second template requires a (permissible) conversion from array to pointer, and that conversion is considered as an exact match for function-matching purposes (§ 6.6.1, p. 245). The nontemplate version is viable but requires a user-defined conversion. That function is less good than an exact match, leaving the two templates as the possible functions to call. As before, the `T*` version is more specialized and is the one that will be selected.

If we want to handle character pointers as strings, we can define two more nontemplate overloads:

```
// convert the character pointers to string and call the string version of debug_rep
string debug_rep(char *p)
{
    return debug_rep(string(p));
}
string debug_rep(const char *p)
{
    return debug_rep(string(p));
}
```

## Missing Declarations Can Cause the Program to Misbehave

It is worth noting that for the `char*` versions of `debug_rep` to work correctly, a declaration for `debug_rep(const string&)` must be in scope when these functions are defined. If not, the wrong version of `debug_rep` will be called:



```

template <typename T> string debug_rep(const T &t);
template <typename T> string debug_rep(T *p);
// the following declaration must be in scope
// for the definition of debug_rep(char*) to do the right thing
string debug_rep(const string &);
string debug_rep(char *p)
{
    // if the declaration for the version that takes a const string& is not in scope
    // the return will call debug_rep(const T&) with T instantiated to string
    return debug_rep(string(p));
}

```

Ordinarily, if we use a function that we forgot to declare, our code won't compile. Not so with functions that overload a template function. If the compiler can instantiate the call from the template, then the missing declaration won't matter. In this example, if we forget to declare the version of `debug_rep` that takes a string, the compiler will *silently* instantiate the template version that takes a `const T&`.



Declare every function in an overload set before you define any of the functions. That way you don't have to worry whether the compiler will instantiate a call before it sees the function you intended to call.

## EXERCISES SECTION 16.3

**Exercise 16.48:** Write your own versions of the `debug_rep` functions.

**Exercise 16.49:** Explain what happens in each of the following calls:

```

template <typename T> void f(T);
template <typename T> void f(const T*);
template <typename T> void g(T);
template <typename T> void g(T*);
int i = 42, *p = &i;
const int ci = 0, *p2 = &ci;
g(42);   g(p);   g(ci);   g(p2);
f(42);   f(p);   f(ci);   f(p2);

```

**Exercise 16.50:** Define the functions from the previous exercise so that they print an identifying message. Run the code from that exercise. If the calls behave differently from what you expected, make sure you understand why.

## 16.4 Variadic Templates

A **variadic template** is a template function or class that can take a varying number of parameters. The varying parameters are known as a **parameter pack**. There are two kinds of parameter packs: A **template parameter pack** represents zero or



more template parameters, and a **function parameter pack** represents zero or more function parameters.

We use an ellipsis to indicate that a template or function parameter represents a pack. In a template parameter list, `class...` or `typename...` indicates that the following parameter represents a list of zero or more types; the name of a type followed by an ellipsis represents a list of zero or more nontype parameters of the given type. In the function parameter list, a parameter whose type is a template parameter pack is a function parameter pack. For example:

```
// Args is a template parameter pack; rest is a function parameter pack
// Args represents zero or more template type parameters
// rest represents zero or more function parameters
template <typename T, typename... Args>
void foo(const T &t, const Args& ... rest);
```

declares that `foo` is a variadic function that has one type parameter named `T` and a template parameter pack named `Args`. That pack represents zero or more additional type parameters. The function parameter list of `foo` has one parameter, whose type is a `const &` to whatever type `T` has, and a function parameter pack named `rest`. That pack represents zero or more function parameters.

As usual, the compiler deduces the template parameter types from the function's arguments. For a variadic template, the compiler also deduces the number of parameters in the pack. For example, given these calls:

```
int i = 0; double d = 3.14; string s = "how now brown cow";
foo(i, s, 42, d);           // three parameters in the pack
foo(s, 42, "hi");           // two parameters in the pack
foo(d, s);                  // one parameter in the pack
foo("hi");                  // empty pack
```

the compiler will instantiate four different instances of `foo`:

```
void foo(const int&, const string&, const int&, const double&);
void foo(const string&, const int&, const char(&)[3]);
void foo(const double&, const string&);
void foo(const char(&)[3]);
```

In each case, the type of `T` is deduced from the type of the first argument. The remaining arguments (if any) provide the number of, and types for, the additional arguments to the function.

## The `sizeof...` Operator

When we need to know how many elements there are in a pack, we can use the `sizeof...` operator. Like `sizeof` (§ 4.9, p. 156), `sizeof...` returns a constant expression (§ 2.4.4, p. 65) and does not evaluate its argument:

```
template<typename ... Args> void g(Args ... args) {
    cout << sizeof...(Args) << endl; // number of type parameters
    cout << sizeof...(args) << endl; // number of function parameters
}
```

**EXERCISES SECTION 16.4**

**Exercise 16.51:** Determine what `sizeof... (Args)` and `sizeof... (rest)` return for each call to `foo` in this section.

**Exercise 16.52:** Write a program to check your answer to the previous question.

**16.4.1 Writing a Variadic Function Template**

In § 6.2.6 (p. 220) we saw that we can use an `initializer_list` to define a function that can take a varying number of arguments. However, the arguments must have the same type (or types that are convertible to a common type). Variadic functions are used when we know neither the number nor the types of the arguments we want to process. As an example, we'll define a function like our earlier `error_msg` function, only this time we'll allow the argument types to vary as well. We'll start by defining a variadic function named `print` that will print the contents of a given list of arguments on a given stream.

Variadic functions are often recursive (§ 6.3.2, p. 227). The first call processes the first argument in the pack and calls itself on the remaining arguments. Our `print` function will execute this way—each call will print its second argument on the stream denoted by its first argument. To stop the recursion, we'll also need to define a nonvariadic `print` function that will take a stream and an object:

```
// function to end the recursion and print the last element
// this function must be declared before the variadic version of print is defined
template<typename T>
ostream &print(ostream &os, const T &t)
{
    return os << t; // no separator after the last element in the pack
}

// this version of print will be called for all but the last element in the pack
template <typename T, typename... Args>
ostream &print(ostream &os, const T &t, const Args&... rest)
{
    os << t << ", "; // print the first argument
    return print(os, rest...); // recursive call; print the other arguments
}
```

The first version of `print` stops the recursion and prints the last argument in the initial call to `print`. The second, variadic, version prints the argument bound to `t` and calls itself to print the remaining values in the function parameter pack.

The key part is the call to `print` inside the variadic function:

```
return print(os, rest...); // recursive call; print the other arguments
```

The variadic version of our `print` function takes three parameters: an `ostream&`, a `const T&`, and a parameter pack. Yet this call passes only two arguments. What happens is that the first argument in `rest` gets bound to `t`. The remaining arguments in `rest` form the parameter pack for the next call to `print`. Thus, on

each call, the first argument in the pack is removed from the pack and becomes the argument bound to `t`. That is, given:

```
print(cout, i, s, 42); // two parameters in the pack
```

the recursion will execute as follows:

| Call                                                   | t | rest... |
|--------------------------------------------------------|---|---------|
| print(cout, i, s, 42)                                  | i | s, 42   |
| print(cout, s, 42)                                     | s | 42      |
| print(cout, 42) calls the nonvariadic version of print |   |         |

The first two calls can match only the variadic version of `print` because the nonvariadic version isn't viable. These calls pass four and three arguments, respectively, and the nonvariadic `print` takes only two arguments.

For the last call in the recursion, `print(cout, 42)`, both versions of `print` are viable. This call passes exactly two arguments, and the type of the first argument is `ostream&`. Thus, the nonvariadic version of `print` is viable.

The variadic version is also viable. Unlike an ordinary argument, a parameter pack can be empty. Hence, the variadic version of `print` can be instantiated with only two parameters: one for the `ostream&` parameter and the other for the `const T&` parameter.

Both functions provide an equally good match for the call. However, a nonvariadic template is more specialized than a variadic template, so the nonvariadic version is chosen for this call (§ 16.3, p. 695).



**WARNING**

A declaration for the nonvariadic version of `print` must be in scope when the variadic version is defined. Otherwise, the variadic function will recurse indefinitely.

EXERCISES SECTION 16.4.1

**Exercise 16.53:** Write your own version of the `print` functions and test them by printing one, two, and five arguments, each of which should have different types.

**Exercise 16.54:** What happens if we call `print` on a type that doesn't have an `<<` operator?

**Exercise 16.55:** Explain how the variadic version of `print` would execute if we declared the nonvariadic version of `print` after the definition of the variadic version.



16.4.2 Pack Expansion

Aside from taking its size, the only other thing we can do with a parameter pack is to **expand** it. When we expand a pack, we also provide a **pattern** to be used on

each expanded element. Expanding a pack separates the pack into its constituent elements, applying the pattern to each element as it does so. We trigger an expansion by putting an ellipsis (...) to the right of the pattern.

For example, our print function contains two expansions:

```
template <typename T, typename... Args>
ostream &
print(ostream &os, const T &t, const Args&... rest) // expand Args
{
    os << t << ", ";
    return print(os, rest...); // expand rest
}
```

The first expansion expands the template parameter pack and generates the function parameter list for print. The second expansion appears in the call to print. That pattern generates the argument list for the call to print.

The expansion of Args applies the pattern const Args& to each element in the template parameter pack Args. The expansion of this pattern is a comma-separated list of zero or more parameter types, each of which will have the form const *type*&. For example:

```
print(cout, i, s, 42); // two parameters in the pack
```

The types of the last two arguments along with the pattern determine the types of the trailing parameters. This call is instantiated as

```
ostream&
print(ostream&, const int&, const string&, const int&);
```

The second expansion happens in the (recursive) call to print. In this case, the pattern is the name of the function parameter pack (i.e., rest). This pattern expands to a comma-separated list of the elements in the pack. Thus, this call is equivalent to

```
print(os, s, 42);
```

## Understanding Pack Expansions

The expansion of the function parameter pack in print just expanded the pack into its constituent parts. More complicated patterns are also possible when we expand a function parameter pack. For example, we might write a second variadic function that calls debug\_rep (§ 16.3, p. 695) on each of its arguments and then calls print to print the resulting strings:

```
// call debug_rep on each argument in the call to print
template <typename... Args>
ostream &errorMsg(ostream &os, const Args&... rest)
{
    // print(os, debug_rep(a1), debug_rep(a2), ..., debug_rep(an))
    return print(os, debug_rep(rest)...);
}
```

The call to `print` uses the pattern `debug_rep(rest)`. That pattern says that we want to call `debug_rep` on each element in the function parameter pack `rest`. The resulting expanded pack will be a comma-separated list of calls to `debug_rep`. That is, a call such as

```
errorMsg(cerr, fcnName, code.num(), otherData, "other", item);
```

will execute as if we had written

```
print(cerr, debug_rep(fcnName), debug_rep(code.num()),
      debug_rep(otherData), debug_rep("otherData"),
      debug_rep(item));
```

In contrast, the following pattern would fail to compile:

```
// passes the pack to debug_rep; print(os, debug_rep(a1, a2, ..., an))
print(os, debug_rep(rest...)); // error: no matching function to call
```

The problem here is that we expanded `rest` in the call to `debug_rep`. This call would execute as if we had written

```
print(cerr, debug_rep(fcnName, code.num(),
                     otherData, "otherData", item));
```

In this expansion, we attempted to call `debug_rep` with a list of five arguments. There is no version of `debug_rep` that matches this call. The `debug_rep` function is not variadic and there is no version of `debug_rep` that has five parameters.



The pattern in an expansion applies separately to each element in the pack.

## EXERCISES SECTION 16.4.2

**Exercise 16.56:** Write and test a variadic version of `errorMsg`.

**Exercise 16.57:** Compare your variadic version of `errorMsg` to the `error_msg` function in § 6.2.6 (p. 220). What are the advantages and disadvantages of each approach?



## 16.4.3 Forwarding Parameter Packs

Under the new standard, we can use variadic templates together with `forward` to write functions that pass their arguments unchanged to some other function. To illustrate such functions, we'll add an `emplace_back` member to our `StrVec` class (§ 13.5, p. 526). The `emplace_back` member of the library containers is a variadic member template (§ 16.1.4, p. 673) that uses its arguments to construct an element directly in space managed by the container.

Our version of `emplace_back` for `StrVec` will also have to be variadic, because `string` has a number of constructors that differ in terms of their parameters.

Because we'd like to be able to use the `string` move constructor, we'll also need to preserve all the type information about the arguments passed to `emplace_back`.

As we've seen, preserving type information is a two-step process. First, to preserve type information in the arguments, we must define `emplace_back`'s function parameters as rvalue references to a template type parameter (§ 16.2.7, p. 693):

```
class StrVec {
public:
    template <class... Args> void emplace_back(Args&&...);
    // remaining members as in § 13.5 (p. 526)
};
```

The pattern in the expansion of the template parameter pack, `&&`, means that each function parameter will be an rvalue reference to its corresponding argument.

Second, we must use `forward` to preserve the arguments' original types when `emplace_back` passes those arguments to `construct` (§ 16.2.7, p. 694):

```
template <class... Args>
inline
void StrVec::emplace_back(Args&&... args)
{
    chk_n_alloc(); // reallocates the StrVec if necessary
    alloc.construct(first_free++, std::forward<Args>(args)...);
}
```

The body of `emplace_back` calls `chk_n_alloc` (§ 13.5, p. 526) to ensure that there is enough room for an element and calls `construct` to create an element in the `first_free` spot. The expansion in the call to `construct`:

```
std::forward<Args>(args)...
```

expands both the template parameter pack, `Args`, and the function parameter pack, `args`. This pattern generates elements with the form

```
std::forward<Ti>(ti)
```

where  $T_i$  represents the type of the  $i$ th element in the template parameter pack and  $t_i$  represents the  $i$ th element in the function parameter pack. For example, assuming `svec` is a `StrVec`, if we call

```
svec.emplace_back(10, 'c'); // adds ccccccccc as a new last element
```

the pattern in the call to `construct` will expand to

```
std::forward<int>(10), std::forward<char>(c)
```

By using `forward` in this call, we guarantee that if `emplace_back` is called with an rvalue, then `construct` will also get an rvalue. For example, in this call:

```
svec.emplace_back(s1 + s2); // uses the move constructor
```

the argument to `emplace_back` is an rvalue, which is passed to `construct` as

```
std::forward<string>(string("the end"))
```

The result type from `forward<string>` is `string&&`, so `construct` will be called with an rvalue reference. The `construct` function will, in turn, forward this argument to the `string` move constructor to build this element.

**ADVICE: FORWARDING AND VARIADIC TEMPLATES**

Variadic functions often forward their parameters to other functions. Such functions typically have a form similar to our `emplace_back` function:

```
// fun has zero or more parameters each of which is
// an rvalue reference to a template parameter type
template<typename... Args>
void fun(Args&&... args) // expands Args as a list of rvalue references
{
    // the argument to work expands both Args and args
    work(std::forward<Args>(args) ...);
}
```

Here we want to forward all of `fun`'s arguments to another function named `work` that presumably does the real work of the function. Like our call to `construct` inside `emplace_back`, the expansion in the call to `work` expands both the template parameter pack and the function parameter pack.

Because the parameters to `fun` are rvalue references, we can pass arguments of any type to `fun`; because we use `std::forward` to pass those arguments, all type information about those arguments will be preserved in the call to `work`.

**EXERCISES SECTION 16.4.3**

**Exercise 16.58:** Write the `emplace_back` function for your `StrVec` class and for the `Vec` class that you wrote for the exercises in § 16.1.2 (p. 668).

**Exercise 16.59:** Assuming `s` is a string, explain `svec.emplace_back(s)`.

**Exercise 16.60:** Explain how `make_shared` (§ 12.1.1, p. 451) works.

**Exercise 16.61:** Define your own version of `make_shared`.



## 16.5 Template Specializations

It is not always possible to write a single template that is best suited for every possible template argument with which the template might be instantiated. In some cases, the general template definition is simply wrong for a type: The general definition might not compile or might do the wrong thing. At other times, we may be able to take advantage of some specific knowledge to write more efficient code than would be instantiated from the template. When we can't (or don't want to) use the template version, we can define a specialized version of the class or function template.

Our `compare` function is a good example of a function template for which the general definition is not appropriate for a particular type, namely, character pointers. We'd like `compare` to compare character pointers by calling `strcmp` rather than by comparing the pointer values. Indeed, we have already overloaded the `compare` function to handle character string literals (§ 16.1.1, p. 654):



```
// first version; can compare any two types
template <typename T> int compare(const T&, const T&);

// second version to handle string literals
template<size_t N, size_t M>
int compare(const char (&)[N], const char (&)[M]);
```

However, the version of `compare` that has two nontype template parameters will be called only when we pass a string literal or an array. If we call `compare` with character pointers, the first version of the template will be called:

```
const char *p1 = "hi", *p2 = "mom";
compare(p1, p2);           // calls the first template
compare("hi", "mom");     // calls the template with two nontype parameters
```

There is no way to convert a pointer to a reference to an array, so the second version of `compare` is not viable when we pass `p1` and `p2` as arguments.

To handle character pointers (as opposed to arrays), we can define a **template specialization** of the first version of `compare`. A specialization is a separate definition of the template in which one or more template parameters are specified to have particular types.

## Defining a Function Template Specialization

When we specialize a function template, we must supply arguments for every template parameter in the original template. To indicate that we are specializing a template, we use the keyword `template` followed by an empty pair of angle brackets (`< >`). The empty brackets indicate that arguments will be supplied for all the template parameters of the original template:

```
// special version of compare to handle pointers to character arrays
template <>
int compare(const char* const &p1, const char* const &p2)
{
    return strcmp(p1, p2);
}
```

The hard part in understanding this specialization is the function parameter types. When we define a specialization, the function parameter type(s) must match the corresponding types in a previously declared template. Here we are specializing:

```
template <typename T> int compare(const T&, const T&);
```

in which the function parameters are references to a `const` type. As with type aliases, the interaction between template parameter types, pointers, and `const` can be surprising (§ 2.5.1, p. 68).

We want to define a specialization of this function with `T` as `const char*`. Our function requires a reference to the `const` version of this type. The `const` version of a pointer type is a constant pointer as distinct from a pointer to `const` (§ 2.4.2, p. 63). The type we need to use in our specialization is `const char* const &`, which is a reference to a `const` pointer to `const char`.

## Function Overloading versus Template Specializations

When we define a function template specialization, we are essentially taking over the job of the compiler. That is, we are supplying the definition to use for a specific instantiation of the original template. It is important to realize that a specialization is an instantiation; it is not an overloaded instance of the function name.



Specializations instantiate a template; they do not overload it. As a result, specializations do not affect function matching.

Whether we define a particular function as a specialization or as an independent, nontemplate function can impact function matching. For example, we have defined two versions of our `compare` function template, one that takes references to array parameters and the other that takes `const T&`. The fact that we also have a specialization for character pointers has no impact on function matching. When we call `compare` on a string literal:

```
compare("hi", "mom")
```

both function templates are viable and provide an equally good (i.e., exact) match to the call. However, the version with character array parameters is more specialized (§ 16.3, p. 695) and is chosen for this call.

Had we defined the version of `compare` that takes character pointers as a plain nontemplate function (rather than as a specialization of the template), this call would resolve differently. In this case, there would be three viable functions: the two templates and the nontemplate character-pointer version. All three are also equally good matches for this call. As we've seen, when a nontemplate provides an equally good match as a function template, the nontemplate is selected (§ 16.3, p. 695)

### KEY CONCEPT: ORDINARY SCOPE RULES APPLY TO SPECIALIZATIONS

In order to specialize a template, a declaration for the original template must be in scope. Moreover, a declaration for a specialization must be in scope before any code uses that instantiation of the template.

With ordinary classes and functions, missing declarations are (usually) easy to find—the compiler won't be able to process our code. However, if a specialization declaration is missing, the compiler will usually generate code using the original template. Because the compiler can often instantiate the original template when a specialization is missing, errors in declaration order between a template and its specializations are easy to make but hard to find.

It is an error for a program to use a specialization and an instantiation of the original template with the same set of template arguments. However, it is an error that the compiler is unlikely to detect.



Templates and their specializations should be declared in the same header file. Declarations for all the templates with a given name should appear first, followed by any specializations of those templates.

## Class Template Specializations

In addition to specializing function templates, we can also specialize class templates. As an example, we'll define a specialization of the library hash template that we can use to store `Sales_data` objects in an unordered container. By default, the unordered containers use `hash<key_type>` (§ 11.4, p. 444) to organize their elements. To use this default with our own data type, we must define a specialization of the hash template. A specialized hash class must define

- An overloaded call operator (§ 14.8, p. 571) that returns a `size_t` and takes an object of the container's key type
- Two type members, `result_type` and `argument_type`, which are the return and argument types, respectively, of the call operator
- The default constructor and a copy-assignment operator (which can be implicitly defined (§ 13.1.2, p. 500))

The only complication in defining this hash specialization is that when we specialize a template, we must do so in the same namespace in which the original template is defined. We'll have more to say about namespaces in § 18.2 (p. 785). For now, what we need to know is that we can add members to a namespace. To do so, we must first open the namespace:

```
// open the std namespace so we can specialize std::hash
namespace std {
} // close the std namespace; note: no semicolon after the close curly
```

Any definitions that appear between the open and close curlyes will be part of the `std` namespace.

The following defines a specialization of hash for `Sales_data`:

```
// open the std namespace so we can specialize std::hash
namespace std {
    template <> // we're defining a specialization with
    struct hash<Sales_data> // the template parameter of Sales_data
    {
        // the type used to hash an unordered container must define these types
        typedef size_t result_type;
        typedef Sales_data argument_type; // by default, this type needs ==
        size_t operator()(const Sales_data& s) const;
        // our class uses synthesized copy control and default constructor
    };
    size_t
    hash<Sales_data>::operator()(const Sales_data& s) const
    {
        return hash<string>()(s.bookNo) ^
            hash<unsigned>()(s.units_sold) ^
            hash<double>()(s.revenue);
    }
} // close the std namespace; note: no semicolon after the close curly
```

Our `hash<Sales_data>` definition starts with `template<>`, which indicates that we are defining a fully specialized template. The template we're specializing is named `hash` and the specialized version is `hash<Sales_data>`. The members of the class follow directly from the requirements for specializing `hash`.

As with any other class, we can define the members of a specialization inside the class or out of it, as we did here. The overloaded call operator must define a hashing function over the values of the given type. This function is required to return the same result every time it is called for a given value. A good hash function will (almost always) yield different results for objects that are not equal.

Here, we delegate the complexity of defining a good hash function to the library. The library defines specializations of the `hash` class for the built-in types and for many of the library types. We use an (unnamed) `hash<string>` object to generate a hash code for `bookNo`, an object of type `hash<unsigned>` to generate a hash from `units_sold`, and an object of type `hash<double>` to generate a hash from `revenue`. We exclusive OR (§ 4.8, p. 154) these results to form an overall hash code for the given `Sales_data` object.

It is worth noting that we defined our hash function to hash all three data members so that our hash function will be compatible with our definition of `operator==` for `Sales_data` (§ 14.3.1, p. 561). By default, the unordered containers use the specialization of `hash` that corresponds to the `key_type` along with the equality operator on the key type.

Assuming our specialization is in scope, it will be used automatically when we use `Sales_data` as a key to one of these containers:

```
// uses hash<Sales_data> and Sales_data operator== from § 14.3.1 (p. 561)
unordered_multiset<Sales_data> SDset;
```

Because `hash<Sales_data>` uses the private members of `Sales_data`, we must make this class a friend of `Sales_data`:

```
template <class T> class std::hash; // needed for the friend declaration
class Sales_data {
friend class std::hash<Sales_data>;
    // other members as before
};
```

Here we say that the specific instantiation of `hash<Sales_data>` is a friend. Because that instantiation is defined in the `std` namespace, we must remember to that this `hash` type is defined in the `std` namespace. Hence, our friend declaration refers to `std::hash`.



To enable users of `Sales_data` to use the specialization of `hash`, we should define this specialization in the `Sales_data` header.

## Class-Template Partial Specializations

Differently from function templates, a class template specialization does not have to supply an argument for every template parameter. We can specify some, but not all, of the template parameters or some, but not all, aspects of the parameters.

A class template **partial specialization** is itself a template. Users must supply arguments for those template parameters that are not fixed by the specialization.



We can partially specialize only a class template. We cannot partially specialize a function template.

In § 16.2.3 (p. 684) we introduced the library `remove_reference` type. That template works through a series of specializations:

```
// original, most general template
template <class T> struct remove_reference {
    typedef T type;
};

// partial specializations that will be used for lvalue and rvalue references
template <class T> struct remove_reference<T&> // lvalue references
{ typedef T type; };

template <class T> struct remove_reference<T&&> // rvalue references
{ typedef T type; };
```

The first template defines the most general version. It can be instantiated with any type; it uses its template argument as the type for its member named `type`. The next two classes are partial specializations of this original template.

Because a partial specialization is a template, we start, as usual, by defining the template parameters. Like any other specialization, a partial specialization has the same name as the template it specializes. The specialization's template parameter list includes an entry for each template parameter whose type is not completely fixed by this partial specialization. After the class name, we specify arguments for the template parameters we are specializing. These arguments are listed inside angle brackets following the template name. The arguments correspond positionally to the parameters in the original template.

The template parameter list of a partial specialization is a subset of, or a specialization of, the parameter list of the original template. In this case, the specializations have the same number of parameters as the original template. However, the parameter's type in the specializations differ from the original template. The specializations will be used for lvalue and rvalue reference types, respectively:

```
int i;
// decltype(42) is int, uses the original template
remove_reference<decltype(42)>::type a;

// decltype(i) is int&, uses first (T&) partial specialization
remove_reference<decltype(i)>::type b;

// decltype(std::move(i)) is int&&, uses second (i.e., T&&) partial specialization
remove_reference<decltype(std::move(i))>::type c;
```

All three variables, `a`, `b`, and `c`, have type `int`.

## Specializing Members but Not the Class

Rather than specializing the whole template, we can specialize just specific member function(s). For example, if `Foo` is a template class with a member `Bar`, we can

specialize just that member:

```
template <typename T> struct Foo {
    Foo(const T &t = T()): mem(t) { }
    void Bar() { /* ... */ }
    T mem;
    // other members of Foo
};

template<>                // we're specializing a template
void Foo<int>::Bar() // we're specializing the Bar member of Foo<int>
{
    // do whatever specialized processing that applies to ints
}
```

Here we are specializing just one member of the `Foo<int>` class. The other members of `Foo<int>` will be supplied by the `Foo` template:

```
Foo<string> fs; // instantiates Foo<string>::Foo()
fs.Bar();      // instantiates Foo<string>::Bar()
Foo<int> fi;    // instantiates Foo<int>::Foo()
fi.Bar();      // uses our specialization of Foo<int>::Bar()
```

When we use `Foo` with any type other than `int`, members are instantiated as usual. When we use `Foo` with `int`, members other than `Bar` are instantiated as usual. If we use the `Bar` member of `Foo<int>`, then we get our specialized definition.

## EXERCISES SECTION 16.5

**Exercise 16.62:** Define your own version of `hash<Sales_data>` and define an `unordered_multiset` of `Sales_data` objects. Put several transactions into the container and print its contents.

**Exercise 16.63:** Define a function template to count the number of occurrences of a given value in a vector. Test your program by passing it a vector of doubles, a vector of ints, and a vector of strings.

**Exercise 16.64:** Write a specialized version of the template from the previous exercise to handle `vector<const char*>` and a program that uses this specialization.

**Exercise 16.65:** In § 16.3 (p. 698) we defined overloaded two versions of `debug_rep` one had a `const char*` and the other a `char*` parameter. Rewrite these functions as specializations.

**Exercise 16.66:** What are the advantages and disadvantages of overloading these `debug_rep` functions as compared to defining specializations?

**Exercise 16.67:** Would defining these specializations affect function matching for `debug_rep`? If so, how? If not, why not?

---

## CHAPTER SUMMARY

---

Templates are a distinctive feature of C++ and are fundamental to the library. A template is a blueprint that the compiler uses to generate specific class types or functions. This process is called instantiation. We write the template once, and the compiler instantiates the template for the type(s) or value(s) with which we use the template.

We can define both function templates and class templates. The library algorithms are function templates and the library containers are class templates.

An explicit template argument lets us fix the type or value of one or more template parameters. Normal conversions are applied to parameters that have an explicit template argument.

A template specialization is a user-provided instantiation of a template that binds one or more template parameters to specified types or values. Specializations are useful when there are types that we cannot use (or do not want to use) with the template definition.

A major part of the latest release of the C++ standard is variadic templates. A variadic template can take a varying number and types of parameters. Variadic templates let us write functions, such as the container `emplace` members and the library `make_shared` function, that pass arguments to an object's constructor.

---

## DEFINED TERMS

---

**class template** Definition from which specific classes can be instantiated. Class templates are defined using the `template` keyword followed by a comma-separated list of one or more template parameters enclosed in `<` and `>` brackets, followed by a class definition.

**default template arguments** A type or a value that a template uses if the user does not supply a corresponding template argument.

**explicit instantiation** A declaration that supplies explicit arguments for all the template parameters. Used to guide the instantiation process. If the declaration is `extern`, the template will not be instantiated; otherwise, the template is instantiated with the specified arguments. There must be a `nonextern` explicit instantiation somewhere in the program for every `extern` template declaration.

**explicit template argument** Template argument supplied by the user in a call to a

function or when defining a template class type. Explicit template arguments are supplied inside angle brackets immediately following the template's name.

**function parameter pack** Parameter pack that represents zero or more function parameters.

**function template** Definition from which specific functions can be instantiated. A function template is defined using the `template` keyword followed by a comma-separated list of one or more template parameters enclosed in `<` and `>` brackets, followed by a function definition.

**instantiate** Compiler process whereby the actual template argument(s) are used to generate a specific instance of the template in which the parameter(s) are replaced by the corresponding argument(s). Functions are instantiated automatically based on the arguments used in a call. We must supply explicit template arguments whenever we use a class template.



**instantiation** Class or function generated by the compiler from a template.

**member template** Member function that is a template. A member template may not be virtual.

**nontype parameter** A template parameter that represents a value. Template arguments for nontype template parameters must be constant expressions.

**pack expansion** Process by which a parameter pack is replaced by the corresponding list of its elements.

**parameter pack** Template or function parameter that represents zero or more parameters.

**partial specialization** Version of a class template in which some but not all of the template parameters are specified or in which one or more parameters are not completely specified.

**pattern** Defines the form of each element in an expanded parameter pack.

**template argument** Type or value used to instantiate a template parameter.

**template argument deduction** Process by which the compiler determines which function template to instantiate. The compiler examines the types of the arguments that were specified using a template parameter. It automatically instantiates a version of the function with those types or values bound to the template parameters.

**template parameter** Name specified in the template parameter list that may be used inside the definition of a template. Template parameters can be type or nontype parameters. To use a class template, we must supply explicit arguments for each template parameter. The compiler uses those types or

values to instantiate a version of the class in which uses of the parameter(s) are replaced by the actual argument(s). When a function template is used, the compiler deduces the template arguments from the arguments in the call and instantiates a specific function using the deduced template arguments.

**template parameter list** List of parameters, separated by commas, to be used in the definition or declaration of a template. Each parameter may be a type or nontype parameter.

**template parameter pack** Parameter pack that represents zero or more template parameters.

**template specialization** Redefinition of a class template, a member of a class template, or a function template, in which some (or all) of the template parameters are specified. A template specialization may not appear until after the base template that it specializes has been declared. A template specialization must appear before any use of the template with the specialized arguments. Each template parameter in a function template must be completely specialized.

**type parameter** Name used in a template parameter list to represent a type. Type parameters are specified following the keyword `typename` or `class`.

**type transformation** Class templates defined by the library that transform their given template type parameter to a related type.

**variadic template** Template that takes a varying number of template arguments. A template parameter pack is specified using an elipsis (e.g., `class...`, `typename...`, or `type-name...`).



# PART IV

## ADVANCED TOPICS

### CONTENTS

---

|                                                       |     |
|-------------------------------------------------------|-----|
| Chapter 17 Specialized Library Facilities . . . . .   | 717 |
| Chapter 18 Tools for Large Programs . . . . .         | 771 |
| Chapter 19 Specialized Tools and Techniques . . . . . | 819 |

Part IV covers additional features that, although useful in the right context, are not needed by every C++ programmer. These features divide into two clusters: those that are useful for large-scale problems and those that are applicable to specialized problems rather than general ones. Features for specialized problems occur both in the language, the topic of Chapter 19, and in the library, Chapter 17.

In Chapter 17 we cover four special-purpose library facilities: the `bitset` class and three new library facilities: `tuples`, regular expressions, and random numbers. We'll also look at some of the less commonly used parts of the IO library.

Chapter 18 covers exception handling, namespaces, and multiple inheritance. These features tend to be most useful in the context of large-scale problems.

Even programs simple enough to be written by a single author can benefit from exception handling, which is why we introduced the basics of exception handling in Chapter 5. However, the need to deal with run-time errors tends to be more important and harder to manage in problems that require large programming teams. In Chapter 18 we review some additional useful exception-handling facilities. We also look in more detail at how exceptions are handled, and show how we can define and use our own exception classes. This section will also cover improvements from the new standard regarding specifying that a particular function will not throw.

Large-scale applications often use code from multiple independent vendors. Combining independently developed libraries would be difficult (if not impossible) if vendors had to put the names they define into a single namespace. Independently developed libraries would almost inevitably use names in common with one another; a name defined in one library would conflict with the use of that name in another library. To avoid name collisions, we can define names inside a namespace.

Whenever we use a name from the standard library, we are using a name defined in the namespace named `std`. Chapter 18 shows how we can define our own namespaces.

Chapter 18 closes by looking at an important but infrequently used language feature: multiple inheritance. Multiple inheritance is most useful for fairly complicated inheritance hierarchies.

Chapter 19 covers several specialized tools and techniques that are applicable to particular kinds of problems. Among the features covered in this chapter are how to redefine how memory allocation works; C++ support for run-time type identification (RTTI), which let us determine the actual type of an expression at run time; and how we can define and use pointers to class members. Pointers to class members differ from pointers to ordinary data or functions. Ordinary pointers only vary based on the type of the object or function. Pointers to members must also reflect the class to which the member belongs. We'll also look at three additional aggregate types: unions, nested classes, and local classes. The chapter closes by looking briefly at a collection of features that are inherently nonportable: the `volatile` qualifier, bit-fields, and linkage directives.